Generazione di gloss ASL da testo inglese: un'analisi neuro-simbolica

Fondamenti teorici, implementazione e risultati negativi con meccanismo

Progetto `neuro_symbolic_t2g`

3 novembre 2026

Sommario

Questa relazione documenta un sistema di traduzione automatica da inglese scritto a *gloss* della lingua dei segni americana (ASL), realizzato con un modello linguistico di piccole dimensioni (Qwen2.5-0.5B-Instruct) addestrato con *supervised fine-tuning* (SFT) e con *reinforcement learning* nella variante GRPO, sotto vincolo simbolico di decodifica.

Il risultato principale non è un miglioramento delle metriche, ma un **risultato negativo con meccanismo**: sul corpus impiegato (ASLG-PC12) le metriche di sovrapposizione lessicale sono *sature*, nel senso preciso che una baseline a regole di circa quindici righe di codice raggiunge ROUGE-L 0,9685 e supera ogni risultato ottenuto con reinforcement learning. Ne segue che la domanda di ricerca inizialmente posta — "il GRPO migliora la generazione di gloss?" — non è ben posta su questo corpus, e che i numeri pubblicati in letteratura su ASLG-PC12 vanno riletti alla luce di tale saturazione.

La stessa pipeline è poi portata su tre compiti con margine reale: la traduzione in gloss della lingua dei segni tedesca (PHOENIX-2014T), il riconoscimento di entità (CoNLL-2003) e la classificazione gerarchica di abstract scientifici (WOS-46985), ciascuno con un vincolo di uscita proprio. Il risultato trasversale è che il GRPO *rinforza* una struttura che il modello già produce, ma non la scopre: senza Trie e senza esempi nel prompt la validity resta bassa su tutti e tre (al più 0,46, e instabile fra semi); e che, con un vincolo attivo, il GRPO trova e sfrutta i varchi che la grammatica lascia aperti.

La relazione è scritta in modo autocontenuto: ogni nozione teorica di complessità media viene introdotta prima di essere usata.

Indice

1 Introduzione **5**
- 1.1 Il problema: dal testo al gloss . . . 5
- 1.2 Che cosa è stato costruito . . . 5
- 1.3 Il risultato principale, in anticipo . . . 6
- 1.4 Oltre il gloss ASL . . . 7
- 1.5 Contributi . . . 7
- 1.6 Come leggere questa relazione . . . 8

2 Fondamenti teorici **9**
- 2.1 Tokenizzazione: il testo come sequenza di simboli . . . 9
- 2.2 Modelli linguistici autoregressivi . . . 9
  - 2.2.1 Architettura: Transformer, in breve . . . 10
- 2.3 Addestramento supervisionato: entropia incrociata . . . 10
  - 2.3.1 Un obiettivo più debole: le etichette parziali . . . 11
- 2.4 Adattamento efficiente: LoRA e quantizzazione . . . 11
  - 2.4.1 LoRA: adattamento a rango basso . . . 11
  - 2.4.2 QLoRA: pesi quantizzati . . . 12
- 2.5 Generazione: strategie di decodifica . . . 12
- 2.6 Strutture dati simboliche: il Trie . . . 12
- 2.7 Apprendimento per rinforzo: impostazione . . . 13
  - 2.7.1 Il gradiente di policy . . . 13
  - 2.7.2 Baseline e vantaggio . . . 13
  - 2.7.3 Vincolo di prossimità e regolarizzazione KL . . . 14
  - 2.7.4 Perché la reward va progettata con sospetto . . . 14
- 2.8 Nozioni di teoria dell'informazione . . . 14
- 2.9 Metriche di valutazione: definizioni . . . 14
- 2.10 Riepilogo della notazione . . . 15

3 Il corpus: ASLG-PC12 **16**
- 3.1 Provenienza e struttura . . . 16
  - 3.1.1 Dimensioni e suddivisione . . . 16
- 3.2 Il corpus è sintetico . . . 17
  - 3.2.1 Evidenza dalla letteratura . . . 17
  - 3.2.2 Evidenza interna: misure sul corpus . . . 17
  - 3.2.3 Quanta incertezza resta? Misura entropica . . . 17
- 3.3 Artefatti del generatore . . . 18
- 3.4 La baseline a regole . . . 18
  - 3.4.1 Costruzione . . . 18
  - 3.4.2 Risultati . . . 19
  - 3.4.3 Il controllo minimale: solo maiuscolo . . . 19

3.5 Conseguenze per il disegno sperimentale . . . 19

4 Modello e stack software **21**
4.1 Il modello base . . . 21
4.2 Adattamento: LoRA e quantizzazione . . . 21
4.3 Lo stack software . . . 22
4.4 Il formato del prompt . . . 22
4.4.1 Zero-shot . . . 22
4.4.2 Few-shot con retrieval . . . 22
4.4.3 Perché questa distinzione è centrale . . . 22
4.5 Il vocabolario di uscita . . . 23

5 Supervised fine-tuning **24**
5.1 L'obiettivo, in concreto . . . 24
5.2 Configurazione . . . 24
5.3 Risultati . . . 25
5.4 Il confronto che conta: SFT contro regola . . . 25
5.5 Che cosa impara l'SFT che la regola non sa . . . 25
5.6 L'SFT è al tetto del corpus, sulle metriche di sovrapposizione . . . 26

6 GRPO: apprendimento per rinforzo di gruppo **27**
6.1 Dall'attore-critico al gruppo . . . 27
6.1.1 La conseguenza immediata: i gruppi morti . . . 27
6.2 Configurazione impiegata . . . 28
6.2.1 Un dato di contesto: l'esposizione ai dati . . . 28
6.3 L'obiettivo ottimizzato . . . 29
6.4 La misura decisiva: il supporto dei rollout . . . 29
6.4.1 La conclusione meccanica . . . 29
6.4.2 Un precedente indipendente . . . 30

7 Il disegno delle reward **31**
7.1 Il problema di progettazione . . . 31
7.2 Lo stack di sette componenti . . . 31
7.3 Componenti sature . . . 32
7.4 Una componente opzionale: `edit_validity` . . . 32
7.4.1 Definizione . . . 32
7.4.2 Test avversario 1: la reward premia la verbosità? . . . 32
7.4.3 Test avversario 2: la calibrazione di w . . . 32

8 Decodifica vincolata: il componente simbolico **33**
8.1 Il problema: dal vincolo sulle parole al vincolo sui token . . . 33
8.2 Implementazione . . . 33
8.2.1 La doppia radice . . . 33
8.2.2 Rapporto con lo stato dell'arte . . . 34
8.2.3 Numeri della costruzione . . . 34
8.3 Il risultato controintuitivo . . . 35
8.3.1 Come si spiega . . . 35
8.3.2 L'effetto veramente importante: il vincolo garantisce la struttura . . . 35
8.4 Il costo . . . 36
8.5 Che cosa non fa il vincolo . . . 36
8.6 Da dove viene il vocabolario, e il tetto che impone . . . 37

8.7 Il costo nascosto della generazione vincolata . . . 37
8.8 Evidenza ablativa . . . 38

9 Obiettivi ausiliari **39**
9.1 Obiettivo 1: la massa ammessa . . . 39
9.1.1 Formulazione . . . 39
9.1.2 Che cosa questo obiettivo può e non può fare . . . 39
9.1.3 Un dettaglio sul gradiente, non catturato dalla previsione . . . 40
9.2 Obiettivo 2: la loss strutturata . . . 40
9.2.1 Formulazione . . . 40
9.2.2 Qualificazione: quanto informa realmente il gloss precedente? . . . 40
9.2.3 Il caveat, dichiarato . . . 41
9.3 Il contratto di non regressione . . . 41
9.4 Il controllo con transizioni mescolate . . . 41
9.5 Risultati . . . 42

10 Metriche di valutazione **43**
10.1 Il difetto di ROUGE-L, misurato . . . 43
10.1.1 Il maiuscolo non conta . . . 43
10.1.2 I prefissi grammaticali vengono distrutti . . . 43
10.1.3 La conclusione operativa . . . 44
10.2 Le metriche primarie . . . 44
10.2.1 Exact match normalizzato . . . 44
10.2.2 Non-copy-token accuracy . . . 44
10.3 Validity . . . 45
10.4 Pass@k . . . 45
10.5 BLEU, chrF e gloss-F1 . . . 45
10.6 Incertezza: due fonti, nessuna delle due quantificata . . . 46
10.7 Il protocollo di valutazione, riassunto . . . 46

11 Risultati **47**
11.1 Provenienza dei numeri . . . 47
11.2 La tabella principale . . . 47
11.3 Osservazione 1: nessun risultato con RL supera la regola . . . 49
11.4 Osservazione 2: l'SFT è al tetto del corpus . . . 49
11.5 Osservazione 3: il vincolo simbolico e le due metriche . . . 49
11.5.1 Una conferma indipendente dalla scelta della metrica . . . 50
11.6 Osservazione 4: le lunghezze non mostrano deriva verso la verbosità . . . 50
11.7 Osservazione 5: il GRPO non degrada una policy già convergente . . . 51
11.8 Osservazione 6: gli obiettivi ausiliari . . . 52
11.8.1 La massa ammessa . . . 52
11.8.2 La loss strutturata . . . 53
11.8.3 Il controllo mescolato: il criterio 2 non è soddisfatto . . . 53
11.9 La decomposizione dell'effetto . . . 54
11.10 Determinismo delle esecuzioni . . . 54
11.11 Riepilogo per non-copy accuracy . . . 55

12 Oltre ASLG-PC12: altri dataset e altri compiti **56**
12.1 I tre compiti . . . 56
12.1.1 PHOENIX-2014T: gloss annotati da umani . . . 56
12.1.2 CoNLL-2003: riconoscimento di entità . . . 57
12.1.3 WOS-46985: classificazione gerarchica . . . 57
12.2 Un vincolo per ogni compito . . . 58
12.2.1 Ogni entità una sola volta . . . 58
12.3 Adattamenti della reward . . . 59
12.4 Protocollo . . . 59
12.5 Risultati . . . 60
12.5.1 PHOENIX-2014T . . . 60
12.5.2 CoNLL-2003 . . . 61
12.5.3 Quanto pesano i semi . . . 62
12.5.4 WOS-46985 . . . 62
12.6 Che cosa dicono i tre dataset insieme . . . 63
12.6.1 Senza Trie e senza esempi, il GRPO trova il formato a fatica . . . 63
12.6.2 Con un vincolo attivo, il GRPO cerca il varco . . . 63
12.6.3 SFT→GRPO è la cella migliore ovunque, ma il guadagno è piccolo . . . 63

13 Organizzazione del codice **65**
13.1 Struttura dei moduli . . . 65
13.2 Il sistema di configurazione . . . 65
13.3 Costruzione centralizzata del prompt . . . 66
13.4 La matrice sperimentale . . . 66
13.5 La validazione delle configurazioni . . . 67
13.6 Le diagnostiche di Markov . . . 67
13.7 Ambiente di esecuzione . . . 67
13.8 Il servizio remoto . . . 67
13.9 La suite di test . . . 67

14 Conclusioni **68**
14.1 Che cosa è stato stabilito . . . 68
14.1.1 Le metriche su ASLG-PC12 sono sature . . . 68
14.1.2 Che cosa compra il vincolo simbolico . . . 68
14.1.3 La posizione onesta sul GRPO . . . 69
14.1.4 Gli obiettivi ausiliari . . . 69
14.1.5 Che cosa aggiungono gli altri dataset . . . 69
14.2 Lavoro futuro . . . 69
14.2.1 Cambiare corpus . . . 69
14.2.2 Ridisegnare l'esperimento primario . . . 70
14.2.3 Consolidare la massa ammessa . . . 70
14.2.4 Il reinforcement learning, se e quando . . . 70
14.3 La lezione trasferibile . . . 70

Capitolo 1

Introduzione

1.1 Il problema: dal testo al gloss

La lingua dei segni americana (*American Sign Language*, ASL) è una lingua naturale completa, con una propria grammatica, che non coincide con l'inglese. Non esiste una forma scritta ufficiale dell'ASL. Per lavorare con strumenti testuali si usa quindi una convenzione di trascrizione chiamata *gloss*: ogni segno viene rappresentato dalla parola inglese in maiuscolo che gli corrisponde più da vicino, eventualmente con marcatori aggiuntivi.

Un esempio reale tratto dal corpus impiegato in questo lavoro:

Inglese	`they will not flinch .`
Gloss	`X-Y WILL DESC-NOT FLINCH .`
Inglese	`this kind of thing must never happen again .`
Gloss	`THIS KIND THING MUST DESC-NEVER HAPPEN DESC-AGAIN .`

Va detto subito, perché condiziona tutta l'interpretazione dei risultati: il gloss **non è** la lingua dei segni. È una trascrizione lessicale impoverita, che perde la struttura spaziale, i marcatori non manuali (espressioni facciali, direzione dello sguardo) e la simultaneità tipica del segnato. Parte della letteratura sostiene che il gloss sia una rappresentazione inadeguata: Yin e Read [52] osservano che nei loro esperimenti la traduzione a partire dal *video* batte quella a partire dal gloss di riferimento, e concludono che il gloss è una rappresentazione inefficiente della lingua dei segni. Sul piano più generale del trattamento delle lingue dei segni nell'elaborazione del linguaggio naturale, Yin et al. [51] raccolgono le raccomandazioni metodologiche del settore, fra cui l'avvertenza a non trattare il gloss come se fosse la lingua.

Nonostante questi limiti il gloss resta l'interfaccia testuale su cui si costruiscono i sistemi esistenti: esistono baseline aperte basate su gloss per la traduzione da lingua parlata a lingua dei segni [24] e architetture di produzione che trattano il compito come selezione e riordino di segni [44].

Il compito affrontato qui, detto *text-to-gloss* (T2G), è dunque:

$$\text{inglese scritto} \longmapsto \text{sequenza di gloss ASL} \tag{1.1}$$

Si tratta di un passo intermedio in una ipotetica pipeline più ampia (testo → gloss → avatar segnante), non di un sistema di traduzione completo.

1.2 Che cosa è stato costruito

Il sistema si basa su un modello linguistico di piccole dimensioni, `Qwen2.5-0.5B-Instruct`, adattato al compito con due famiglie di metodi:

- **SFT** (*supervised fine-tuning*): addestramento supervisionato classico, in cui il modello impara a riprodurre il gloss di riferimento;
- **GRPO** (*Group Relative Policy Optimization*): apprendimento per rinforzo, in cui il modello genera più candidati per ogni frase e viene premiato in base alla qualità relativa di ciascuno.

Le due famiglie sono state combinate in quattro celle sperimentali: modello base senza addestramento, solo SFT, solo GRPO a partire dal modello base, e la pipeline completa SFT seguito da GRPO.

A queste si sovrappongono due fattori ortogonali:

- la modalità di *prompting*: *zero-shot* (solo la frase da tradurre) oppure *few-shot* con retrieval (nel prompt vengono inseriti $k = 3$ esempi simili recuperati dal training set);
- la *decodifica vincolata*: un componente simbolico (un *Trie*, ossia un albero di prefissi) che a ogni passo di generazione vieta al modello di emettere token che non portino a un gloss valido.

Quest'ultimo elemento è ciò che rende il sistema *neuro-simbolico*: una rete neurale produce le probabilità, una struttura dati simbolica ne restringe il supporto.

1.3 Il risultato principale, in anticipo

Questa relazione non riporta un miglioramento delle metriche. Riporta un **risultato negativo con meccanismo**, che è più utile di un miglioramento apparente perché spiega *perché* la domanda iniziale era mal posta.

Il fatto centrale, misurato e documentato nel Capitolo 3, è che il corpus impiegato (ASLG-PC12) non è stato prodotto da annotatori umani: è **sintetico**, generato applicando regole all'inglese. La conseguenza pratica è che una baseline a regole, costruita in circa quindici righe di codice contando le corrispondenze parola-per-parola nel solo training set, ottiene:

Sistema	ROUGE-L	Exact match
Baseline a regole (nessun apprendimento)	0,9685	0,5865
SFT (modello addestrato)	0,9681	0,7662
GRPO da modello base (few-shot)	0,6082	0,0544
SFT seguito da GRPO (few-shot)	0,9628	0,7296
SFT con obiettivo di massa ammessa	0,9769	0,8341

Si legga con attenzione la terza riga: il miglior risultato ottenuto con reinforcement learning a partire dal modello base (0,6082) sta *molto sotto* una regola che non apprende nulla (0,9685). Persino la trasformazione banale "metti tutto in maiuscolo" raggiunge ROUGE-L 0,6454, cioè batte il GRPO. Sul ROUGE-L, inoltre, l'SFT non si distingue dalla regola (0,9681 contro 0,9685): il suo unico vantaggio netto è sull'exact match (0,7662 contro 0,5865).

Da qui seguono tre conseguenze che strutturano il resto della relazione:

1. **Nessun numero è interpretabile senza la baseline a regole accanto.** Un risultato di 0,6 su questo corpus non è un successo parziale: è un fallimento rispetto a un riferimento che nessuno aveva calcolato.

2. **La domanda "il GRPO aiuta il T2G?" non è ben posta su ASLG-PC12.** Il compito è quasi deterministico, l'SFT lo satura, e qualsiasi metodo di rinforzo parte già in svantaggio rispetto a una regola. La domanda si riduce a "il GRPO memorizza una mappa quasi-identità meglio dell'SFT?", che riguarda l'ottimizzazione e non la lingua dei segni.

3. **Le metriche di sovrapposizione premiano la copia.** Poiché il gloss è in larga parte l'inglese maiuscolizzato, ROUGE-L e le metriche analoghe assegnano punteggi alti a un modello che si limita a copiare l'ingresso (Capitolo 10).

1.4 Oltre il gloss ASL

Il text-to-gloss su ASLG-PC12 è il banco di prova di una domanda più generale: se il GRPO possa allineare un modello a produrre *uscite strutturate che rispettano vincoli rigidi*, con o senza supporti (esempi nel prompt, Trie in decodifica, SFT preliminare). Poiché ASLG-PC12 è saturo, il Capitolo 12 porta la stessa pipeline su tre compiti in cui il margine esiste e la struttura dell'uscita cambia:

Compito	Ingresso	Uscita
gloss DGS (PHOENIX)	`am donnerstag regen ...`	`DONNERSTAG REGEN ...`
entità (CoNLL)	`Dominick Gauthier ( Canada )`	`PER:Dominick_Gauthier LOC:Canada`
classificazione (WOS)	abstract scientifico	`CS Software_engineering`

Ciascuno ha un vincolo di uscita proprio: il vocabolario chiuso per il gloss, un Trie costruito dalla frase stessa per le entità, le sole coppie (dominio, area) ammesse per la classificazione.

1.5 Contributi

Al netto del risultato negativo, il lavoro produce alcuni contributi verificabili:

- una **baseline a regole** riutilizzabile che rende interpretabili i numeri su questo corpus, insieme a una metrica insensibile alla copia (*non-copy-token accuracy*);
- un **censimento degli artefatti** del generatore del corpus: il 10,47% delle righe di training contiene parole corrotte da una rimozione di sottostringa (per esempio `therefore` diventa `DESC-REFORE`);
- la **misura dei gruppi a varianza nulla** durante l'addestramento, che spiega perché il GRPO non sposta una policy SFT già convergente: fra il 61% e il 71% dei gruppi non produce gradiente (Capitolo 6);
- un risultato **neuro-simbolico non ovvio**: il vincolo simbolico peggiora la metrica di sovrapposizione e contemporaneamente *garantisce* la struttura, che il GRPO da solo non impara: senza Trie solo il 19% delle uscite è fatto interamente di gloss del vocabolario (Capitolo 8);
- due **obiettivi ausiliari** qualificati prima di essere addestrati (Capitolo 9): l'obiettivo di massa ammessa funziona (SFT con massa ammessa: ROUGE-L 0,9769, exact match 0,8341, contro 0,9681 e 0,7662 dell'SFT semplice), mentre l'obiettivo strutturato fallisce il proprio controllo (0,9430 e 0,6053);

- su tre dataset aggiuntivi (Capitolo 12), la misura di **quando il GRPO impara una struttura senza Trie**: con esempi nel prompt sì (validity 0,99 su PHOENIX-2014T), senza a fatica (validity fra 0 e 0,46, e instabile fra semi);
- un caso documentato di **reward hacking sotto vincolo**: su CoNLL-2003 il GRPO impara a ripetere ogni entità con tutti i tipi, e bloccato quello a scrivere entità annidate; la correzione va nella grammatica, non nella reward;
- un risultato negativo sull'**esplorazione**: su CoNLL una reward che premia solo l'entità col tipo giusto non fa imparare i tipi al GRPO dal modello base, che non campiona abbastanza tipi giusti da rinforzarli.

1.6 Come leggere questa relazione

Il Capitolo 2 introduce le nozioni teoriche necessarie (modelli linguistici autoregressivi, entropia incrociata, adattamento a rango basso, gradiente di policy) e può essere saltato da chi le conosce. I capitoli successivi seguono la pipeline: dati (Capitolo 3), modello (Capitolo 4), i due metodi di addestramento (Capitoli 5 e 6), il disegno delle reward (Capitolo 7), il componente simbolico (Capitolo 8), gli obiettivi ausiliari (Capitolo 9), le metriche (Capitolo 10) e i risultati (Capitolo 11). Il Capitolo 12 estende l'analisi a PHOENIX-2014T, CoNLL-2003 e WOS-46985. Gli ultimi due capitoli riguardano l'organizzazione del codice e le conclusioni. I valori numerici riportati sono tutti misurati.

Capitolo 2

Fondamenti teorici

Questo capitolo introduce da zero le nozioni necessarie a leggere il resto della relazione. Chi ha già familiarità con modelli linguistici autoregressivi, adattamento a rango basso e gradiente di policy può passare direttamente al Capitolo 3.

L'ordine è quello delle dipendenze logiche: prima come si rappresenta il testo (tokenizzazione), poi che cosa è un modello linguistico e come si addestra (entropia incrociata), poi come si adatta un modello grande con poche risorse (LoRA e quantizzazione), poi come si genera testo (decodifica), infine come si addestra con un segnale di qualità invece che con un riferimento (reinforcement learning e gradiente di policy).

2.1 Tokenizzazione: il testo come sequenza di simboli

Un modello linguistico non lavora su caratteri né su parole, ma su *token*: elementi di un vocabolario finito $\mathcal{V}$ costruito una volta per tutte a partire da un grande corpus. Un token può essere una parola intera, un frammento di parola, un segno di punteggiatura o uno spazio.

Formalmente, il tokenizzatore è una funzione

$$\mathrm{tok} : \Sigma^* \longrightarrow \mathcal{V}^* \tag{2.1}$$

dove Σ è l'alfabeto dei caratteri e $\mathcal{V}^*$ l'insieme delle sequenze finite di token. Si richiede che tok sia invertibile (*lossless*): esiste detok tale che $\mathrm{detok}(\mathrm{tok}(s)) = s$ per ogni stringa s.

Questo dettaglio, apparentemente pedante, è centrale in questo progetto. La parola gloss `DESC-NOT` non è un token: viene spezzata in più token, per esempio `DESC`, `-`, `NOT`. Ne segue che un vincolo espresso a livello di *parola* ("emetti solo gloss appartenenti al vocabolario") deve essere tradotto in un vincolo a livello di *token* ("emetti solo token che siano prefisso valido di almeno un gloss"). È esattamente il problema che il Trie del Capitolo 8 risolve.

Un secondo dettaglio: molti tokenizzatori moderni codificano lo spazio come parte del token successivo. Il token che rappresenta `"HOUSE"` a inizio frase e quello che rappresenta `" HOUSE"` in mezzo a una frase sono identificatori *diversi*. Anche questo ha una conseguenza diretta (la doppia radice del Trie, sezione 8.2.1).

2.2 Modelli linguistici autoregressivi

Sia $y = (y_1, \ldots, y_T)$ una sequenza di token. Un modello linguistico *autoregressivo* fattorizza la probabilità congiunta della sequenza usando la regola della catena:

$$p_\theta(y) \;=\; \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}), \qquad y_{<t} = (y_1, \ldots, y_{t-1}). \tag{2.2}$$

Nel caso condizionale che ci interessa (dato un ingresso x, l'inglese, produrre un'uscita y, il gloss) la fattorizzazione diventa

$$p_\theta(y \mid x) \;=\; \prod_{t=1}^{T} p_\theta(y_t \mid x, y_{<t}). \tag{2.3}$$

Il modello calcola, a ogni passo t, un vettore di *logits* $z_t \in \mathbb{R}^{|\mathcal{V}|}$, cioè un punteggio non normalizzato per ogni token del vocabolario. La distribuzione di probabilità si ottiene applicando la funzione softmax:

$$p_\theta(y_t = v \mid x, y_{<t}) \;=\; \frac{\exp(z_{t,v})}{\sum_{u \in \mathcal{V}} \exp(z_{t,u})}. \tag{2.4}$$

Il denominatore, detto *funzione di partizione*, garantisce che le probabilità sommino a uno. Il fatto che la somma sia su *tutto* il vocabolario è ciò che rende non banale imporre un vincolo: se si vietano alcuni token, il denominatore va ricalcolato sull'insieme ammesso.

2.2.1 Architettura: Transformer, in breve

L'architettura impiegata (Qwen2.5, si veda il Capitolo 4) è un Transformer *decoder-only*. Non è necessario, per leggere questa relazione, conoscerne i dettagli; bastano tre fatti.

1. Ogni token viene trasformato in un vettore (*embedding*) e passa attraverso una pila di blocchi identici nella struttura.
2. Ogni blocco contiene un modulo di *attenzione*, che mette in relazione la posizione corrente con tutte le posizioni precedenti, e un modulo *feed-forward* (una rete densa a due strati con non linearità). I moduli dell'attenzione sono chiamati per convenzione `q_proj`, `k_proj`, `v_proj`, `o_proj`; quelli feed-forward `gate_proj`, `up_proj`, `down_proj`. Sono esattamente i sette moduli su cui agisce l'adattamento LoRA descritto più avanti.
3. L'attenzione è *causale*: la posizione t non può guardare le posizioni $> t$. È questa mascheratura che rende valida la fattorizzazione (2.3) e che permette di calcolare, in un unico passaggio, le probabilità di tutti i token di una sequenza nota.

2.3 Addestramento supervisionato: entropia incrociata

Dato un insieme di esempi $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, l'obiettivo dell'addestramento supervisionato è massimizzare la log-verosimiglianza dei riferimenti, ossia minimizzare la *loss* di entropia incrociata:

$$\mathcal{L}_{\mathrm{CE}}(\theta) \;=\; -\frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T_i} \log p_\theta\Big(y_t^{(i)} \,\Big|\, x^{(i)}, y_{<t}^{(i)}\Big)\,. \tag{2.5}$$

Interpretazione: per ogni posizione si guarda quale probabilità il modello assegna al token *corretto* e si penalizza il logaritmo negativo di quella probabilità. Se il modello assegna probabilità 1 al token corretto il contributo è 0; se assegna probabilità prossima a 0 il contributo diverge.

Due proprietà rilevanti nel seguito:

Teacher forcing. Nella (2.5) il condizionamento è sul *prefisso di riferimento* $y_{<t}^{(i)}$, non su quanto il modello avrebbe generato. Questo rende l'addestramento parallelizzabile su tutte le posizioni, ma crea una discrepanza fra addestramento e generazione (*exposure bias*): in generazione il modello condiziona sui propri errori.

Mascheratura del prompt. Nel nostro caso interessa solo che il modello impari a produrre il gloss, non a riprodurre l'inglese di ingresso. Le posizioni corrispondenti al prompt vengono quindi *escluse* dalla somma. Nella pratica implementativa si assegna a quelle posizioni l'etichetta convenzionale -100, che le librerie usate interpretano come "ignora" (si veda il Capitolo 5).

2.3.1 Un obiettivo più debole: le etichette parziali

Esiste una variante utile quando non si conosce il token corretto, ma si conosce un *insieme* $\mathcal{A}_t \subseteq \mathcal{V}$ che lo contiene. In quel caso si può massimizzare la probabilità che il modello assegni all'insieme, invece che a un singolo elemento:

$$\mathcal{L}_{\mathrm{AM}}(\theta) \;=\; -\log \sum_{v \in \mathcal{A}_t} p_\theta(v \mid x, y_{<t}). \tag{2.6}$$

Questa è la formulazione classica dell'apprendimento a etichette parziali (*partial-label learning*, [5]) e coincide con la massimizzazione della verosimiglianza marginale (*maximum marginal likelihood*, [12]). Va sottolineata una proprietà che sarà decisiva nel Capitolo 9: la (2.6) è *piatta* sull'insieme ammesso nel suo *valore*, cioè ridistribuire la massa *interna* ad $\mathcal{A}_t$ non la modifica. Il suo *gradiente*, però, non è piatto: dipende da come la massa è distribuita fra i token dell'insieme. Un effetto sull'exact match, oltre che sulla *calibrazione* (quanta probabilità va sui token legali), non è quindi escluso a priori.

2.4 Adattamento efficiente: LoRA e quantizzazione

Aggiornare tutti i parametri di un modello linguistico richiede memoria per i pesi, per i gradienti e per gli stati dell'ottimizzatore. Due tecniche riducono drasticamente questo costo.

2.4.1 LoRA: adattamento a rango basso

L'idea di LoRA (*Low-Rank Adaptation*, [16]) è che l'aggiornamento necessario per adattare un modello pre-addestrato a un compito specifico abbia *rango intrinseco basso*. Invece di aggiornare una matrice di pesi $W_0 \in \mathbb{R}^{d \times k}$, si congela W_0 e si apprende un aggiornamento fattorizzato:

$$W \;=\; W_0 \;+\; \Delta W, \qquad \Delta W \;=\; \frac{\alpha}{r}\, BA, \tag{2.7}$$

con $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$ e $r \ll \min(d, k)$. Il numero di parametri addestrabili passa da dk a $r(d + k)$.

I due iperparametri hanno un significato preciso:

- r (*rango*) fissa la capacità dell'adattamento;
- α (*alpha*) è un fattore di scala; il rapporto α/r nella (2.7) serve a rendere l'ampiezza dell'aggiornamento indipendente dalla scelta di r, così che cambiare r non richieda di ritarare il learning rate.

All'inizializzazione $B = 0$, quindi $\Delta W = 0$ e il modello adattato coincide esattamente con quello di partenza.

2.4.2 QLoRA: pesi quantizzati

La quantizzazione riduce la precisione numerica con cui i pesi congelati sono memorizzati. In QLoRA [7] i pesi base W_0 sono memorizzati a 4 bit, mentre gli adattatori A, B restano in precisione più alta e sono gli unici a ricevere gradiente. Il risultato è che un modello che in precisione a 16 bit non entrerebbe in memoria diventa addestrabile su una sola GPU.

Il costo è un rumore di quantizzazione sui pesi base.

2.5 Generazione: strategie di decodifica

Data la distribuzione (2.4), produrre una sequenza richiede una regola di scelta. Le opzioni rilevanti qui sono tre.

Greedy. Si prende a ogni passo il token più probabile, $y_t = \arg\max_v p_\theta(v \mid x, y_{<t})$. È deterministico ma tende a produrre testo ripetitivo [15, 45].

Campionamento con temperatura. Si campiona da una distribuzione riscalata:

$$p_\theta^{(\tau)}(v) \;\propto\; \exp\!\Big(\frac{z_{t,v}}{\tau}\Big)\,. \tag{2.8}$$

Per $\tau \to 0$ si recupera il greedy; per $\tau = 1$ la distribuzione del modello; per $\tau > 1$ una distribuzione più piatta, quindi più varia. La temperatura è il principale strumento per controllare la diversità dei candidati, e nel reinforcement learning è ciò che determina se il gruppo di candidati contiene o no differenze su cui apprendere.

Decodifica vincolata. Si azzera la probabilità dei token non ammessi prima della normalizzazione. Operativamente si pone $z_{t,v} \leftarrow -\infty$ per $v \notin \mathcal{A}_t$, cosicché

$$p_\theta(v \mid x, y_{<t}, \mathcal{A}_t) = \begin{cases} \dfrac{\exp(z_{t,v})}{\sum_{u \in \mathcal{A}_t} \exp(z_{t,u})} & v \in \mathcal{A}_t, \\ 0 & \text{altrimenti.} \end{cases} \tag{2.9}$$

Questa è la forma di generazione con *lessico chiuso* adottata nel Capitolo 8, ed è imparentata con la decodifica grammaticalmente vincolata [9, 31] e con la decodifica a vincoli lessicali [14, 35].

Va osservato che la (2.9) *non* è una semplice riparametrizzazione: sposta massa di probabilità dai token vietati a quelli ammessi in proporzione ai logits originali. Il modello non "sa" di essere vincolato, dunque il vincolo può produrre sequenze che il modello non avrebbe mai generato liberamente. È il motivo per cui il vincolo può *peggiorare* una metrica di sovrapposizione e contemporaneamente *migliorare* la validità strutturale.

2.6 Strutture dati simboliche: il Trie

Un *Trie* (o albero dei prefissi) è un albero in cui ogni nodo rappresenta un prefisso e ogni arco è etichettato con un simbolo. Dato un vocabolario finito di stringhe G (nel nostro caso: l'insieme dei gloss ammessi), il Trie costruito su G permette di rispondere in tempo proporzionale alla lunghezza del prefisso alla domanda:

> "dato il prefisso di token s, quali token possono seguire senza rendere impossibile il completamento in un elemento di G?"

Formalmente, detto $\mathrm{Pref}(G)$ l'insieme dei prefissi di token degli elementi di G, l'insieme ammesso è

$$\mathcal{A}(s) \;=\; \{\, v \in \mathcal{V} \;:\; s \cdot v \in \mathrm{Pref}(G) \,\} \tag{2.10}$$

dove $\cdot$ denota la concatenazione. Il Trie è precisamente la struttura che materializza $\mathrm{Pref}(G)$ in modo che la (2.10) sia calcolabile con un accesso a un dizionario.

2.7 Apprendimento per rinforzo: impostazione

Nell'addestramento supervisionato si dispone del riferimento $y^{(i)}$. Nel reinforcement learning si dispone invece di un *segnale di qualità*, la *reward*, che valuta un'uscita prodotta dal modello stesso.

Si modella la generazione come un processo decisionale:

- lo *stato* al passo t è il prefisso $(x, y_{<t})$;
- l'*azione* è la scelta del token y_t;
- la *policy* è il modello stesso, $\pi_\theta(y_t \mid x, y_{<t})$;
- la *reward* $R(x, y) \in \mathbb{R}$ è assegnata alla sequenza completa (*reward terminale*).

L'obiettivo è massimizzare la reward attesa:

$$J(\theta) \;=\; \mathbb{E}_{x \sim \mathcal{D}}\, \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)} \big[R(x, y) \big]. \tag{2.11}$$

2.7.1 Il gradiente di policy

Il problema tecnico è che la variabile rispetto a cui si deriva, θ, compare nella distribuzione su cui si prende l'aspettazione. La soluzione classica è l'identità del *likelihood ratio*: poiché $\nabla_\theta \pi_\theta(y) = \pi_\theta(y)\, \nabla_\theta \log \pi_\theta(y)$, si ottiene

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{y \sim \pi_\theta} \big[R(x, y)\, \nabla_\theta \log \pi_\theta(y \mid x) \big]. \tag{2.12}$$

Questo è il teorema del gradiente di policy, alla base di REINFORCE [47]. Il significato è diretto: si aumenta la log-probabilità delle sequenze con reward alta e si diminuisce quella delle sequenze con reward bassa.

2.7.2 Baseline e vantaggio

La stima (2.12) è corretta ma ha varianza elevata. Si dimostra che sottrarre una qualunque quantità b indipendente dall'azione lascia il gradiente inalterato in aspettazione, poiché $\mathbb{E}_{y \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(y)] = 0$. Si definisce allora il *vantaggio*

$$A(x, y) \;=\; R(x, y) - b(x) \tag{2.13}$$

e si sostituisce R con A nella (2.12). La scelta di $b(x)$ è ciò che distingue gli algoritmi fra loro: PPO [38], impiegato anche nella messa a punto di modelli linguistici con feedback umano [29], apprende una rete separata (il *critic*) che stima $b(x)$; GRPO [40] usa invece la media delle reward di un gruppo di candidati generati per lo stesso x, eliminando il critic. Il dettaglio è nel Capitolo 6.

Da (2.13) segue un fatto che ricorrerà come chiave interpretativa di tutta la parte sperimentale: se *tutti* i candidati per un dato x ricevono la stessa reward, allora $A = 0$ per ciascuno e il gradiente è nullo. Quel prompt non contribuisce all'apprendimento. Chiameremo *gruppo morto* un gruppo in questa condizione, e *supporto* la frazione di gruppi che producono un gradiente non nullo.

2.7.3 Vincolo di prossimità e regolarizzazione KL

Aggiornamenti troppo grandi degradano la policy in modo irreversibile. Si introduce quindi un termine che penalizza l'allontanamento da una policy di riferimento π_{ref} (tipicamente il modello prima dell'addestramento per rinforzo):

$$J_\beta(\theta) \;=\; \mathbb{E}\big[A(x,y)\big] \;-\; \beta\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})\,, \tag{2.14}$$

dove $D_{\text{KL}}(p\|q) = \sum_v p(v) \log \frac{p(v)}{q(v)} \geq 0$ è la divergenza di Kullback-Leibler, nulla se e solo se $p = q$. Il coefficiente β regola quanto la policy è libera di allontanarsi dal riferimento.

2.7.4 Perché la reward va progettata con sospetto

Ottimizzare una reward imperfetta produce comportamenti che massimizzano la misura senza migliorare l'obiettivo reale. È il fenomeno noto come *reward hacking* o *specification gaming* [1, 8]; un caso documentato e pertinente è la deriva verso risposte più lunghe quando la reward correla con la lunghezza [41]. La teoria classica del *reward shaping* [26] caratterizza quali trasformazioni della reward preservano la politica ottima; nella pratica di questo progetto la contromisura adottata è diversa e più diretta: ogni componente di reward viene sottoposta a test avversari costruiti a mano (Capitolo 7).

2.8 Nozioni di teoria dell'informazione

Due quantità servono nel seguito per misurare quanta incertezza esista in un compito.

L'*entropia* di una variabile discreta Y misura l'incertezza media, in bit:

$$H(Y) \;=\; -\sum_{y} p(y) \log_2 p(y). \tag{2.15}$$

L'*entropia condizionale* misura l'incertezza residua su Y una volta noto X:

$$H(Y \mid X) \;=\; -\sum_{x,y} p(x,y) \log_2 p(y \mid x). \tag{2.16}$$

La differenza $H(Y) - H(Y \mid X) = I(X;Y)$ è l'informazione mutua: quanta incertezza su Y viene rimossa dall'osservazione di X. Nel Capitolo 3 queste quantità vengono stimate empiricamente sul corpus per rispondere a una domanda concreta: quanta incertezza resta sul gloss una volta noto il token inglese corrispondente? La risposta (0,856 bit su 8,578) quantifica in che senso il compito sia quasi deterministico.

2.9 Metriche di valutazione: definizioni

Le metriche usate nel seguito vanno definite qui, perché la loro forma esatta determina l'interpretazione dei risultati.

Exact match (EM). Frazione di uscite identiche al riferimento dopo normalizzazione. È una metrica binaria e severa, ma non manipolabile.

ROUGE-L. Basata sulla più lunga sottosequenza comune (*longest common subsequence*, LCS) fra generato g e riferimento r [21]. Con $P = \text{LCS}(g,r)/|g|$ e $R = \text{LCS}(g,r)/|r|$, la misura F è

$$F_1 \;=\; \frac{2PR}{P+R}. \tag{2.17}$$

Essendo una sottosequenza (non una sottostringa), l'ordine conta ma non la contiguità.

BLEU. Media geometrica delle precisioni su n-grammi, con una penalità per uscite troppo brevi [30]. Va distinta la variante *a frase* (mediata sulle frasi) da quella *a corpus* (con conteggi aggregati): danno numeri diversi e vanno riportate separatamente.

chrF. Analoga a BLEU ma su n-grammi di *caratteri* [34]; più robusta alla morfologia.

Pass@k. Frazione di prompt per cui almeno una fra k generazioni supera una soglia di correttezza. Detti $g_1^{(i)}, \dots, g_N^{(i)}$ le N generazioni per il prompt i, $r^{(i)}$ il riferimento, θ la soglia e $\mathbb{1}[\cdot]$ la funzione indicatrice, la definizione adottata in questo lavoro è

$$\text{pass@}k \;=\; \frac{1}{M}\sum_{i=1}^{M} \mathbb{1}\Big[\,\exists\, j \le k :\; \text{ROUGE-L}\Big(g_j^{(i)}, r^{(i)}\Big) \ge \theta\Big]\,, \tag{2.18}$$

dove M è il numero di prompt. Si tratta di una *media empirica* sulle prime k generazioni di ciascun prompt, con $\theta = 0{,}3$.

Un avvertimento che il Capitolo 10 svilupperà in dettaglio: tutte le metriche di questa lista, tranne l'exact match, sono metriche di *sovrapposizione*. Su un compito in cui l'uscita corretta è in larga parte una copia dell'ingresso, esse assegnano punteggi alti a sistemi che non hanno imparato nulla. Per questo il progetto introduce una metrica complementare, la *non-copy-token accuracy*, che valuta solo le posizioni in cui il gloss *differisce* dal testo di partenza.

2.10 Riepilogo della notazione

Simbolo	Significato
x	frase inglese di ingresso (prompt)
y	sequenza di gloss generata
$\mathcal{V}$	vocabolario di token del modello
$\mathcal{A}_t$	insieme dei token ammessi al passo t
π_θ	policy corrente (il modello addestrato)
π_{ref}	policy di riferimento (modello prima del RL)
z_t	logits al passo t
$R(x, y)$	reward della sequenza completa
$A(x, y)$	vantaggio
G	numero di candidati per gruppo (GRPO)
r, α	rango e scala di LoRA
β	coefficiente della penalità KL
τ	temperatura di campionamento

Capitolo 3

Il corpus: ASLG-PC12

Questo capitolo è il più importante della relazione. Non perché descriva l'ingegneria più complessa, ma perché ogni conclusione dei capitoli successivi dipende da un fatto stabilito qui: il corpus impiegato non è stato prodotto da annotatori umani, ed è quasi deterministico. Tutte le metriche di sovrapposizione lessicale che si possono calcolare su di esso sono, in senso tecnico, sature.

3.1 Provenienza e struttura

Il corpus è ASLG-PC12 [27], ottenuto allineando il corpus parlamentare inglese con una versione in gloss ASL. Ogni riga è una coppia:

Campo	Esempio
`text`	`they will not flinch .`
`gloss`	`X-Y WILL DESC-NOT FLINCH .`

I marcatori con prefisso hanno una funzione grammaticale: `X-` indica un pronome (`X-Y` per *they*, `X-WE` per *we*, `X-I` per *I*), mentre `DESC-` indica un descrittore, tipicamente un avverbio o una negazione (`DESC-NOT`, `DESC-NEVER`, `DESC-AGAIN`).

3.1.1 Dimensioni e suddivisione

Grandezza	Valore
Righe totali (dato grezzo)	87 710
Suddivisione	90/10, seed 42
Righe di training	72 979
Righe di test	8 109
Vocabolario gloss (token distinti)	16 121
Vocabolario gloss dopo filtraggio	15 472

Le due cifre del vocabolario differiscono perché il filtro scarta forme che non possono essere emesse (si veda il Capitolo 8); l'insieme effettivamente usato dal vincolo di decodifica conta 15 472 gloss.

Il seed 42 e la proporzione 90/10 sono fissati nel codice: ogni numero riportato in questa relazione si riferisce a questa suddivisione.

3.2 Il corpus è sintetico

Questo è il punto centrale. ASLG-PC12 non è stato annotato da segnanti: è stato generato applicando regole all'inglese. La conseguenza è che la relazione fra `text` e `gloss` è in larga parte una trasformazione meccanica invertibile, non una traduzione fra due lingue.

L'evidenza è di due tipi: esterna (letteratura) e interna (misure sul corpus).

3.2.1 Evidenza dalla letteratura

Moryossef et al. [25] documentano la natura sintetica del corpus e il suo uso come dato aumentato piuttosto che come riferimento.

Più diretta è l'analisi di Peng et al. [32]. Nella loro Tabella 7 riportano la sovrapposizione lessicale fra i token del gloss e quelli del testo:

Corpus	Sovrapposizione token gloss/testo
ASLG-PC12	98,23%
PHOENIX-2014T	15,59%

Un fattore sei di differenza. Su ASLG-PC12 il gloss usa quasi esattamente le stesse parole del testo; su un corpus annotato da umani (PHOENIX-2014T, lingua dei segni tedesca) no.

Nella loro Tabella 3, gli stessi autori mostrano che una *regola scritta a mano* ottiene BLEU 96,75 su ASLG-PC12. Un valore di questo ordine su un compito di traduzione automatica non significa che il sistema traduca bene: significa che non c'è traduzione da fare.

3.2.2 Evidenza interna: misure sul corpus

Sono state effettuate misure indipendenti sul dato in nostro possesso, per quantificare esattamente quanto il gloss sia una copia del testo. I risultati:

Misura	Valore
Token gloss identici al token sorgente maiuscolizzato	61,79%
idem, ignorando i prefissi `DESC-` e `X-`	79,74%
idem, confrontando i primi 4 caratteri (*stem*)	90,26%
Righe con `len(text) == len(gloss)` (in token)	27,5%

La lettura della prima colonna è progressiva. Quasi due token su tre sono letteralmente la parola inglese in maiuscolo. Se si perdona il prefisso grammaticale si arriva a quattro su cinque. Se si perdona la desinenza (usando solo la radice di quattro caratteri) si arriva a nove su dieci.

Solo il 27,5% delle righe ha la stessa lunghezza in token nelle due colonne. Questo dato ha un'importanza tecnica specifica: significa che l'allineamento posizionale uno-a-uno fra testo e gloss è disponibile solo su un quarto del corpus. Tutte le analisi che richiedono un allineamento (per esempio la stima delle transizioni nel Capitolo 9) sono limitate a quel sottoinsieme, e questo va tenuto presente come *caveat* nella lettura dei risultati.

3.2.3 Quanta incertezza resta? Misura entropica

La domanda si può porre in modo preciso con gli strumenti della sezione sui fondamenti di teoria dell'informazione. Sia Y il token gloss e X il token sorgente allineato. Le stime empiriche sono:

Quantità	Valore
$H(Y)$, entropia del token gloss	8,578 bit
$H(Y \mid X)$, entropia condizionata al token sorgente	0,856 bit
$I(X;Y)$, informazione mutua	7,722 bit

Cioè: conoscere il token inglese rimuove circa il 90% dell'incertezza sul token gloss. Resta meno di un bit di ambiguità residua per token.

Questa singola riga spiega, da sola, la maggior parte dei risultati sperimentali:

- una tabella di corrispondenze uno-a-uno cattura quasi tutto il segnale (sezione 3.4);
- l'SFT satura il compito rapidamente (Capitolo 5);
- il reinforcement learning non ha spazio per migliorare, perché non c'è una scelta difficile da fare (Capitolo 6).

3.3 Artefatti del generatore

Poiché il corpus è generato da regole, ne eredita i difetti. Il più vistoso è una rimozione di sottostringa applicata senza vincoli di confine di parola: la sequenza di caratteri `the` viene rimossa *anche quando è interna a un'altra parola*.

Parola inglese	Gloss prodotto	Occorrenze (training)
`therefore`	`DESC-REFORE`	1 875
`these`	`SE`	2 847
`there`	`DESC-RE`	3 279

In totale 7 641 righe di training, ossia il 10,47%, contengono almeno una parola corrotta da questo meccanismo. Una riga su dieci del riferimento contiene una forma che nessun segnante userebbe.

Poiché i riferimenti pubblicati in letteratura sono calcolati sul dato così com'è, gli artefatti sono documentati e conservati, in modo che i nostri numeri restino confrontabili con quelli. Costituiscono anzi un argomento a favore della tesi centrale: un corpus con il 10% di righe corrotte da un bug di sostituzione di stringhe non è un riferimento adeguato per valutare la qualità di una traduzione.

3.4 La baseline a regole

Se il corpus è quasi deterministico, la domanda naturale è: quanto arriva lontano una regola? La risposta è il riferimento più importante di tutta la relazione.

3.4.1 Costruzione

Il metodo è elementare:

1. si scorre il *solo training set* e, sulle righe in cui testo e gloss hanno la stessa lunghezza in token, si contano le corrispondenze posizionali parola $\rightarrow$ gloss;
2. per ogni parola inglese si conserva il gloss più frequente, ottenendo un lessico uno-a-uno di 10 996 voci;

3. si apprendono, con lo stesso conteggio, 13 *cancellazioni*: parole inglesi che nel gloss non compaiono (fra cui `the` e `of`);

4. in inferenza si applica il lessico parola per parola, senza guardare il contesto.

Si noti che la baseline è *context-free*: traduce ogni parola indipendentemente dalle altre. Non c'è modello, non c'è addestramento, non ci sono parametri oltre alla tabella.

3.4.2 Risultati

Sul sottoinsieme di 2 000 prompt usato per la valutazione dei modelli, così da consentire il confronto diretto con le tabelle del Capitolo 11:

Metrica	Valore
ROUGE-L	0,9685
Exact match	0,5865
Gloss F1	0,9665
Non-copy-token accuracy	0,9424
BLEU (a corpus)	0,8907
chrF	96,47

3.4.3 Il controllo minimale: solo maiuscolo

Per isolare quanto del punteggio derivi dal semplice fatto che il gloss è maiuscolo, è stata valutata la trasformazione uppercase(text) senza alcun lessico:

Sistema	ROUGE-L
uppercase(text)	0,6454
Baseline a regole	0,9685

Il risultato è duplice. Da un lato, il lessico aggiunge un contributo reale (0,65 $\rightarrow$ 0,97): la baseline non è banale. Dall'altro, e questo è il punto scomodo, un ROUGE-L di 0,6454 è ottenibile *senza fare nulla*, e come si vedrà nel Capitolo 11 supera il miglior risultato ottenuto con reinforcement learning a partire dal modello base (0,6082).

3.5 Conseguenze per il disegno sperimentale

Da questo capitolo derivano quattro vincoli metodologici, che sono stati applicati a tutte le misure riportate nel seguito.

1. **Nessuna metrica va riportata senza la baseline a regole accanto.** La baseline è il livello zero: un sistema che non la supera non ha imparato nulla di utile su questo corpus.

2. **Servono metriche insensibili alla copia.** La non-copy-token accuracy, definita nel Capitolo 10, valuta solo le posizioni in cui il gloss differisce dal testo. È l'unica metrica di sovrapposizione che discrimina fra i sistemi in modo interpretabile.

3. **Il confronto con i numeri pubblicati va contestualizzato.** Il miglior ROUGE-L pubblicato su ASLG-PC12 è 95,87 [32], cioè *sotto* la nostra baseline a regole. Non è un errore degli autori: è una proprietà del corpus.

4. **Le conclusioni non si trasferiscono ad altri corpora.** Su PHOENIX-2014T, dove la sovrapposizione è 15,59%, i migliori risultati T2G riportati sono intorno a ROUGE-L 55,18 [28]. Quello è un compito reale; questo no.

Capitolo 4

Modello e stack software

4.1 Il modello base

Il modello impiegato è `Qwen2.5-0.5B-Instruct`: un Transformer decoder-only da circa 0,5 miliardi di parametri, nella variante già sottoposta a istruzione (*instruct*), cioè addestrata a seguire richieste formulate in linguaggio naturale.

La scelta di un modello piccolo è motivata da tre ragioni, in ordine di peso:

1. **Vincolo di risorse.** L'addestramento gira su una singola GPU di un cluster condiviso con una quota di una sola allocazione per volta. Il reinforcement learning richiede di generare G candidati per ogni prompt a ogni passo, quindi il costo di generazione domina; un modello grande renderebbe l'esperimento infattibile.

2. **Adeguatezza al compito.** Come stabilito nel Capitolo 3, il compito ha meno di un bit di incertezza residua per token. Non è un compito che richiede capacità di ragionamento: richiede di memorizzare una mappa lessicale.

3. **Iterazione.** Un modello piccolo permette cicli di esperimento brevi, che è ciò che serve quando l'oggetto dell'indagine è il *disegno* sperimentale e non la prestazione assoluta.

La variante *instruct* è preferita alla variante base perché il prompt adottato ha forma di istruzione, e perché nella cella *few-shot* gli esempi sono inseriti come turni di conversazione.

4.2 Adattamento: LoRA e quantizzazione

Il modello base non viene aggiornato. Si applica l'adattamento a rango basso descritto nella (2.7) con la configurazione seguente.

Iperparametro	Valore
Rango r	32
Scala α	64
Dropout	0,05
Moduli adattati	7
Quantizzazione dei pesi base	4 bit (QLoRA)

I sette moduli sono `q_proj`, `k_proj`, `v_proj`, `o_proj` (le quattro proiezioni dell'attenzione) e `gate_proj`, `up_proj`, `down_proj` (le tre proiezioni del blocco feed-forward). Adattare anche il blocco feed-forward, e non solo l'attenzione, è la configurazione che la letteratura

su QLoRA [7] indica come preferibile quando il numero di parametri addestrabili non è il vincolo stringente.

Il rapporto $\alpha/r = 2$ è la scelta convenzionale. Il dropout 0,05 agisce solo sul ramo dell'adattatore.

4.3 Lo stack software

La riproducibilità richiede di fissare le versioni. Lo stack è il seguente.

Componente	Versione	Ruolo
PyTorch	`2.7.1+cu118`	tensori, autograd
CUDA	11,8	runtime GPU
Transformers	5,3,0	modello, tokenizzatore, generazione
TRL	0,24,0	`SFTTrainer`, `GRPOTrainer`
PEFT	0,19,1	implementazione LoRA
Unsloth	—	kernel ottimizzati, caricamento 4 bit
torchao	0,17,0	primitive di quantizzazione
sacrebleu	2,6,0	BLEU e chrF
sentence-transformers	5,2,3	retrieval per il few-shot

4.4 Il formato del prompt

Il modello riceve un'istruzione e produce il gloss. Sono state usate due modalità.

4.4.1 Zero-shot

Il prompt contiene solo l'istruzione e la frase da tradurre. La lunghezza massima del prompt è fissata a 256 token.

4.4.2 Few-shot con retrieval

Il prompt contiene, prima della frase da tradurre, $k = 3$ esempi *recuperati* dal training set in base alla somiglianza semantica con la frase corrente. Il recupero è realizzato con `sentence-transformers`: le frasi di training sono codificate in vettori una volta sola, e a inferenza si cercano i tre vicini più prossimi.

La lunghezza massima del prompt in questa modalità è 768 token, per far posto agli esempi.

4.4.3 Perché questa distinzione è centrale

Anticipando il Capitolo 11: la differenza fra zero-shot e few-shot è, numericamente, il fattore *più grande* osservato in tutto l'esperimento. Il modello base con vincolo di decodifica passa da ROUGE-L 0,1373 (zero-shot) a 0,4664 (few-shot), un salto di 0,33 ottenuto *senza aggiornare un solo parametro*. Il miglior risultato con reinforcement learning a partire dal modello base è circa 0,61 (few-shot): rispetto al punto di partenza (0,1373) l'escursione totale è di circa 0,47, e il solo prompting ne fornisce circa il 70%. Ne segue che gran parte dell'effetto attribuibile alla "pipeline" è in realtà effetto di prompting.

4.5 Il vocabolario di uscita

Il modello genera token, ma l'uscita valida è una sequenza di gloss. Il collegamento fra i due livelli è il vocabolario gloss di 15 472 elementi descritto nel Capitolo 3, tradotto in un insieme di 3 967 identificatori di token ammessi come *primo* token di un gloss.

La costruzione, la doppia radice necessaria per gestire il token con spazio iniziale, e le quattro forme gloss che risultano non emettibili sono trattate nel Capitolo 8.

Capitolo 5

Supervised fine-tuning

5.1 L'obiettivo, in concreto

L'addestramento supervisionato minimizza l'entropia incrociata (2.5) sulle sole posizioni della *completion*, cioè del gloss. Le posizioni del prompt sono escluse assegnando loro l'etichetta convenzionale -100, che le librerie impiegate interpretano come "non contribuire alla loss".

Questa distinzione è più importante di quanto sembri. Se il prompt non venisse mascherato, il modello dedicherebbe capacità a riprodurre l'inglese di ingresso, che è già noto e non serve a nulla. Peggio: su questo corpus, dove il gloss è in larga parte una copia maiuscolizzata del testo, addestrare il modello a riprodurre il testo rinforzerebbe esattamente la scorciatoia che si vuole misurare separatamente.

Formalmente, detto $m_t \in \{0, 1\}$ l'indicatore "la posizione t appartiene alla completion", la loss effettivamente ottimizzata è

$$\mathcal{L}_{\mathrm{SFT}}(\theta) \;=\; -\frac{\sum_i \sum_t m_t^{(i)} \log \pi_\theta\left(y_t^{(i)} \mid x^{(i)}, y_{<t}^{(i)}\right)}{\sum_i \sum_t m_t^{(i)}}. \tag{5.1}$$

5.2 Configurazione

Iperparametro	Valore
Epoche	3
Batch per dispositivo	4
Accumulo del gradiente	4
Batch effettivo	16
Learning rate	$2 \cdot 10^{-5}$
Passi di warmup	50
Lunghezza massima di sequenza	768
Holdout interno per la validazione	2%
Pazienza per l'arresto anticipato	3

Il *warmup* porta il learning rate da zero al valore nominale nei primi 50 passi, evitando che i primi aggiornamenti, calcolati su gradienti ancora disallineati, danneggino i pesi. L'*holdout* del 2% è ritagliato dal training set (non dal test) e serve solo a decidere l'arresto anticipato: se la loss di validazione non migliora per 3 valutazioni consecutive, l'addestramento si ferma.

5.3 Risultati

Valutazione su 2 000 prompt del test set, con campionamento a temperatura 0,7 e $N = 5$ generazioni per prompt (Capitolo 10).

Metrica	SFT
ROUGE-L	0,9681
Exact match	0,7662
Gloss F1	0,9698
BLEU (a corpus)	0,9316
chrF (a corpus)	96,36
Validity	1,0000
Non-copy-token accuracy	0,9510

Nudi, questi numeri sembrano un successo pieno. Vanno letti accanto alla baseline a regole.

5.4 Il confronto che conta: SFT contro regola

Metrica	Regola	SFT	Delta
ROUGE-L	0,9685	0,9681	−0,0004
Exact match	0,5865	0,7662	+0,1797
Non-copy-token accuracy	0,9424	0,9510	+0,0086

La *grandezza* del vantaggio dipende radicalmente dalla metrica:

- su ROUGE-L le due grandezze sono indistinguibili: l'SFT sta di 0,0004 *sotto* la regola (con decodifica greedy sale a 0,9696, di poco sopra). Un modello addestrato su 73 mila esempi non guadagna nulla, su questa metrica, rispetto a una tabella di corrispondenze;
- su exact match il guadagno è +0,1797 (0,7662 contro 0,5865), cioè un abbattimento del tasso di errore dal 41% al 23%, pari a una riduzione relativa di circa il 43%. È l'unico vantaggio netto dell'SFT sulla regola.

Il confronto sull'exact match è inoltre sbilanciato a sfavore dell'SFT. Il 7,15% delle frasi valutate contiene un gloss mai visto nel training, che con il Trie non si può generare; la regola invece copia in maiuscolo le parole che non conosce e su quelle frasi arriva al 25%. Sulle 1 857 frasi che entrambi possono tradurre l'SFT ottiene 0,8252 contro 0,6123: 21 punti di vantaggio invece di 18 (sezione 8.6).

Questa discrepanza è la prima manifestazione concreta della saturazione: ROUGE-L, essendo una metrica di sovrapposizione, non ha risoluzione residua per distinguere i due sistemi, mentre l'exact match sì. È un argomento operativo per promuovere l'exact match e la non-copy-token accuracy a metriche primarie (Capitolo 10). Il fatto che l'SFT non superi la regola su ROUGE-L rafforza la tesi del capitolo 3, invece di indebolirla: su questo corpus la sovrapposizione lessicale è quasi interamente spiegata da un lessico parola-per-parola.

5.5 Che cosa impara l'SFT che la regola non sa

Il confronto aggregato dice *quanto*, non *che cosa*. Il vantaggio dell'SFT sull'exact match si concentra in una regolarità linguistica: la copula inglese (*is*, *are*, *be*) talvolta compare

nel gloss come `BE` e talvolta viene omessa, e l'omissione dipende dal contesto. La baseline, essendo context-free, deve scegliere una volta per tutte e sbaglia sistematicamente sull'altro caso; l'SFT, che condiziona sul contesto, no.

Questo delimita il contributo dell'apprendimento su questo corpus: è piccolo ma identificabile, ed è una dipendenza contestuale che un lessico context-free non può esprimere.

5.6 L'SFT è al tetto del corpus, sulle metriche di sovrapposizione

Il ROUGE-L dell'SFT è 0,9681. Il miglior valore pubblicato in letteratura su ASLG-PC12 è 95,87 [32], cioè 0,9587.

Segue che l'SFT di un modello da 0,5 miliardi di parametri con adattatori LoRA *supera* lo stato dell'arte pubblicato su questo corpus. Questo non è un risultato di cui vantarsi: è la prova più chiara che la metrica non misura ciò che si crede misuri. Un modello piccolo, addestrato per poco più di due epoche su una singola GPU, non batte anni di ricerca; batte una metrica satura, e lo fa di misura nulla rispetto a una regola.

Il tetto, tuttavia, riguarda le metriche di sovrapposizione e non l'exact match. Con exact match 0,7662 resta un margine reale: l'obiettivo di massa ammessa lo porta a 0,8341 (Capitolo 9) e la pipeline SFT seguito da GRPO in zero-shot a 0,8095, con ROUGE-L 0,9744. Su ROUGE-L, invece, questi stessi sistemi si muovono in una fascia di circa un centesimo (0,9628–0,9769), troppo stretta per discriminarli.

Capitolo 6

GRPO: apprendimento per rinforzo di gruppo

6.1 Dall'attore-critico al gruppo

Come stabilito nella (2.13), ogni metodo di gradiente di policy richiede una *baseline* $b(x)$ per ridurre la varianza. PPO [38] la ottiene addestrando una seconda rete, il *critic*, che stima il valore atteso della reward a partire dallo stato. Questo raddoppia approssimativamente il costo di memoria e introduce un secondo problema di ottimizzazione.

GRPO (*Group Relative Policy Optimization*, [40, 11]) rimuove il critic con un'idea semplice: se per lo stesso prompt x si generano G candidati, la media delle loro reward è già una stima della baseline. Formalmente, dati i candidati $o_1, \dots, o_G \sim \pi_\theta(\cdot \mid x)$ con reward $R_1, \dots, R_G$:

$$A_i \;=\; R_i - \frac{1}{G}\sum_{j=1}^{G} R_j \qquad \text{(opzionalmente diviso per } \operatorname{std}(R_1, \dots, R_G)\text{).} \tag{6.1}$$

Il gruppo è la baseline. È questa la differenza sostanziale da PPO: non c'è nessuna rete di valore da addestrare, e il segnale di apprendimento è puramente *relativo* all'interno del gruppo.

L'obiettivo completo, includendo la penalità KL (2.14), si scrive

$$J_{\mathrm{GRPO}}(\theta) \;=\; \mathbb{E}\left[\frac{1}{\mathcal{Z}}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\rho_{i,t}(\theta)\,A_i\right] \;-\; \beta\, D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})\,, \tag{6.2}$$

dove $\rho_{i,t}$ è il rapporto di probabilità fra policy corrente e policy di campionamento (con eventuale clipping in stile PPO) e $\mathcal{Z}$ è un *denominatore di normalizzazione* la cui scelta, come si vedrà nella sezione 6.3, non è affatto un dettaglio.

6.1.1 La conseguenza immediata: i gruppi morti

Dalla (6.1) segue un fatto elementare e decisivo. Se tutti i candidati di un gruppo ricevono la stessa reward, allora $R_i = \bar{R}$ per ogni i, dunque $A_i = 0$ per ogni i, dunque il contributo di quel prompt al gradiente è *esattamente nullo*.

Chiameremo:

gruppo a varianza nulla un gruppo in cui $\operatorname{std}(R) = 0$;

gruppo morto un gruppo in cui tutte le reward sono al valore minimo possibile (tipicamente -1), che è un caso particolare del precedente ma qualitativamente peggiore: non solo non c'è segnale, ma la policy non ha nemmeno un candidato parzialmente buono da cui partire;

supporto la frazione di gruppi con $\mathrm{std}(R) > 0$, cioè la frazione di prompt che contribuiscono effettivamente all'apprendimento.

Il supporto è la grandezza diagnostica più importante per decidere se un esperimento di reinforcement learning sia *possibile* prima ancora di chiedersi se sia utile. Se il supporto è vicino a zero, l'algoritmo gira, consuma tempo di calcolo, produce metriche, ma non apprende. La misura del supporto in questo progetto è nella sezione 6.4.

6.2 Configurazione impiegata

Iperparametro	Valore
Candidati per gruppo G	8
Passi di ottimizzazione	2 000
Learning rate	$3 \cdot 10^{-6}$
Passi di warmup	200
Coefficiente KL β	0,04
Temperatura di campionamento τ	0,7
Lunghezza massima della completion	128
Lunghezza massima del prompt (zero-shot)	256
Lunghezza massima del prompt (few-shot)	768
Batch per dispositivo	1
Accumulo del gradiente	8

Il learning rate è quasi un ordine di grandezza più basso di quello dell'SFT ($3 \cdot 10^{-6}$ contro $2 \cdot 10^{-5}$): è la scelta convenzionale nel reinforcement learning su modelli linguistici, dove aggiornamenti aggressivi degradano rapidamente la policy. I resoconti su sistemi di grande scala addestrati con rinforzo [19] confermano che il regime di apprendimento va tenuto conservativo, e che il fattore limitante è tipicamente la qualità del segnale piuttosto che la dimensione del passo.

6.2.1 Un dato di contesto: l'esposizione ai dati

Ogni passo di GRPO consuma *un solo* prompt: gli 8 micro-batch dell'accumulo del gradiente sono gli 8 rollout dello stesso prompt, non 8 prompt distinti. Le celle addestrate con GRPO fanno 5 000 passi, quindi vedono 5 000 prompt, che su un training set di 72 979 esempi corrispondono a

$$\frac{5\,000}{72\,979} \approx 0{,}069 \quad \text{epoche}, \tag{6.3}$$

cioè il 6,9% del training set.

L'SFT si ferma per early stopping a 10 000 passi con batch effettivo 16, cioè circa 2,24 epoche; il GRPO vede meno di un decimo dei dati una sola volta. Questo va detto esplicitamente perché rende improprio qualunque confronto diretto "SFT contro GRPO" interpretato come confronto fra metodi: i due bracci non hanno la stessa esposizione ai dati. Il GRPO non è stato sotto-addestrato per errore, ma per vincolo di budget di calcolo: ogni passo richiede $G = 8$ generazioni complete, quindi un passo di GRPO costa circa un ordine di grandezza più di un passo di SFT.

6.3 L'obiettivo ottimizzato

Il termine $\mathcal{Z}$ nella (6.2) determina come le sequenze di lunghezza diversa vengono pesate fra loro; Dr. GRPO [23] identifica nella normalizzazione per lunghezza una fonte di bias. Tutti gli esperimenti di reinforcement learning di questo lavoro usano i valori predefiniti di TRL 0,24: variante di loss `dapo` (normalizzazione sul numero di token del batch), reward centrate sulla media del gruppo e divise per la sua deviazione standard (`scale_rewards='group'`), senza dynamic sampling né clip-higher. La variante Dr. GRPO compare solo come cella di ablazione.

6.4 La misura decisiva: il supporto dei rollout

Prima di chiedersi se il GRPO migliori le metriche, si può chiedere se abbia *materiale* su cui lavorare. TRL registra durante l'addestramento la frazione di gruppi a varianza nulla; nei log delle run conservate ce ne sono 1 000 valori per run, uno ogni cinque passi.

Cella	Media	Mediana
GRPO dal modello base, zero-shot	0,5%	0%
GRPO dal modello base, few-shot	3,6%	0%
GRPO dal modello base, few-shot, senza Trie	5,6%	0%
SFT seguito da GRPO, zero-shot	70,8%	80%
SFT seguito da GRPO, few-shot	61,0%	60%
SFT seguito da GRPO, rollout a temperatura alta	27,3%	20%

La tabella separa due regimi.

Dal modello base il segnale c'è. Meno del 6% dei gruppi resta senza gradiente, anche senza Trie. Con sette componenti di reward continue, due candidati diversi non ricevono quasi mai la stessa reward: un gruppo ha varianza nulla solo quando gli otto candidati coincidono. Il limite del GRPO dal modello base non è dunque la mancanza di segnale, ma ciò che il segnale può insegnare in 5 000 prompt (0,069 epoche) e il fatto che la reward premi surrogati della traduzione corretta (Capitolo 7).

Dalla policy SFT il segnale manca. Fra il 61% e il 71% dei gruppi ha varianza nulla, perché la policy è già concentrata e gli otto candidati sono la stessa sequenza. Il gradiente è nullo non per fallimento, ma per saturazione.

Il gruppo morto in senso stretto, con tutte le reward al minimo, resta un caso teorico: non si osserva in nessuna run conservata, perché anche il modello base senza vincolo, con gli esempi nel prompt, produce candidati diversi fra loro. Non è stata addestrata una cella zero-shot senza Trie, dove il modello produce inglese (validity 3,7%), quindi quel regime non è misurato.

6.4.1 La conclusione meccanica

Il fallimento misurato è uno solo, quello per saturazione. Quando la policy è già forte gli otto candidati coincidono, $R_i = \bar{R}$ per ogni i e il vantaggio è nullo: è la ragione per cui il GRPO applicato dopo l'SFT lascia il sistema dov'era (sezione 11.7). Aumentare la diversità dei rollout riduce i gruppi senza gradiente (dal 61% al 27% con la temperatura di campionamento alta), ma il risultato finale non cambia: ROUGE-L 0,9634 contro 0,9628. Più segnale non si traduce in una policy migliore quando la policy è già al tetto del corpus.

Va letto con proporzione anche l'altro lato: il GRPO dal modello base (few-shot) impara, portando il ROUGE-L da 0,4664 a 0,6082, pur avendo segnale su quasi ogni gruppo. Resta molto al di sotto sia della baseline a regole (0,9685) sia dell'SFT (0,9681), che però ha visto i dati più di trenta volte tanto.

6.4.2 Un precedente indipendente

Il problema non è specifico di questo progetto. Choshen et al. [4] mostrano che i guadagni attribuiti al RL nella traduzione automatica derivano spesso da un effetto di *peakiness* (massa di probabilità concentrata sui token già più probabili) più che da una reward informativa; Kiegeland e Kreutzer [18] trovano che la riduzione non è universale. Entrambi concordano sulla domanda da porre prima di addestrare: quanto era già corretta la policy di partenza, e la reward ha margine per distinguere fra le sue uscite? Per il RLVR su modelli di ragionamento la letteratura recente è analoga: migliora `pass@1` ma raramente amplia il supporto delle risposte corrette [48, 53], mentre un RL prolungato può scoprire strategie nuove quando la competenza iniziale è sufficiente [22]: il margine deve esistere per cominciare.

Capitolo 7

Il disegno delle reward

7.1 Il problema di progettazione

Nell'addestramento supervisionato l'obiettivo è dato: la (2.5) non richiede scelte. Nel reinforcement learning invece la funzione di reward *è* il progetto: definisce che cosa il modello considererà buono, e ogni sua imperfezione verrà sfruttata dall'ottimizzazione. È il fenomeno del *reward hacking* [1, 8], di cui la deriva verso risposte più lunghe è l'esempio più documentato [41].

La convenzione adottata in questo progetto è che ogni componente di reward produca un valore in $[0, 1]$, che viene poi mappato in $[-1, 1]$ tramite

$$R \;=\; 2s - 1, \qquad s \in [0, 1]. \tag{7.1}$$

La ragione della mappatura è che il vantaggio (6.1) è invariante per traslazione, ma un intervallo centrato rende più leggibile la telemetria: un valore negativo indica "sotto la metà della qualità massima".

7.2 Lo stack di sette componenti

La reward complessiva è una combinazione convessa di sette componenti, con pesi che sommano a 1,0. Lo stack storico comprendeva tre ulteriori componenti basate su percorsi di Viterbi, valutate e scartate perché ridondanti: a parità di lunghezza le tre formule collassano sullo stesso segnale.

Componente	Che cosa misura	Peso
`translation`	somiglianza con il gloss di riferimento	0,20
`bleu`	sovrapposizione di n-grammi	0,20
`gold_structure`	aderenza alla struttura del riferimento	0,20
`gloss_order`	correttezza dell'ordine dei gloss	0,10
`verifier_scaled`	esito di un verificatore esterno	0,10
`format`	conformità al formato di uscita	0,10
`repetition`	assenza di ripetizioni degeneri	0,10
	Totale	**1,00**

Questo stack coincide con la configurazione predefinita del GRPO: non esiste quindi un'ablazione separata per lo "stack storico", che *è* la cella GRPO di base. Una ottava componente, `edit_validity` (descritta nella sezione 7.4), ha peso nullo nella configurazione predefinita ed è studiata solo come ablazione.

7.3 Componenti sature

Le componenti `format` e `repetition` sono praticamente sempre soddisfatte: quasi ogni candidato le rispetta, quindi sono **sature**. Una componente satura ha varianza quasi nulla *all'interno del gruppo*, dunque contribuisce al vantaggio di ogni candidato in misura quasi identica e il suo contributo si cancella nella sottrazione della media ((6.1)). Il 20% del peso complessivo agisce come una costante additiva: hanno senso come rete di sicurezza contro degenerazioni, ma il segnale effettivo viene dalle altre cinque componenti.

7.4 Una componente opzionale: `edit_validity`

`edit_validity` cattura con una sola misura la vicinanza al riferimento e l'appartenenza al vocabolario ammesso. Nella cella di ablazione `edit-validity` è l'unica componente attiva, con peso 1,0: raggiunge ROUGE-L 0,6373 ed exact match 0,0608.

7.4.1 Definizione

$$\texttt{edit_sim} = 1 - \frac{\mathrm{lev}(g, r)}{\max(|g|, |r|)}, \tag{7.2}$$

$$s = (1 - w) \cdot \texttt{edit_sim} \; + \; w \cdot \texttt{frac_in_vocab}, \tag{7.3}$$

$$R = 2s - 1, \tag{7.4}$$

dove lev è la distanza di Levenshtein a livello di token fra generato g e riferimento r, `frac_in_vocab` è la frazione di token generati appartenenti al vocabolario gloss, e $w \in [0, 1]$ è un peso con valore predefinito 0,5.

7.4.2 Test avversario 1: la reward premia la verbosità?

L'ipotesi da escludere è la più classica: che il modello possa aumentare la reward allungando l'uscita [41]. Se il generato coincide con il riferimento e vi si appendono k token spuri, allora lev $= k$, $|g| = G + k$ con $G = |r|$, e

$$\texttt{edit_sim} \; = \; 1 - \frac{k}{G + k}, \tag{7.5}$$

che è *strettamente decrescente* in k: la verbosità non è un canale di attacco per questa reward.

7.4.3 Test avversario 2: la calibrazione di w

Se w è troppo alto, la reward premia l'uso di token in vocabolario indipendentemente dal fatto che formino una sequenza sensata. Con $w = 0{,}5$ un candidato quasi corretto con un token fuori vocabolario ($R = +0{,}500$) batte la ripetizione degenere di token legali ($R = +0{,}125$); con $w = 0{,}75$ l'ordinamento **si inverte** ($+0{,}563$ contro $+0{,}500$). Il valore predefinito è quindi fissato a $w = 0{,}5$. Il controllo non richiede addestramento, solo la valutazione della reward su candidati costruiti a mano.

Capitolo 8

Decodifica vincolata: il componente simbolico

Questo capitolo descrive l'unico elemento propriamente *neuro-simbolico* del sistema, e riporta il risultato più controintuitivo dell'intero lavoro: il vincolo simbolico *peggiora* la metrica di sovrapposizione e contemporaneamente *abilita* il segnale di reward.

8.1 Il problema: dal vincolo sulle parole al vincolo sui token

L'uscita valida è una sequenza di gloss appartenenti a un vocabolario finito G di 15 472 elementi. Il modello, però, genera *token*, e un gloss corrisponde in generale a più token (sezione sui fondamenti di tokenizzazione). Non si può quindi semplicemente restringere il vocabolario di uscita: bisogna tradurre il vincolo sul livello delle parole in un vincolo sul livello dei token.

La formulazione corretta è quella della (2.10): al passo t, dato il prefisso di token già emesso, sono ammessi i token che mantengono il prefisso estendibile a un elemento di G. Il Trie costruito su G è la struttura dati che rende questo calcolo immediato.

Operativamente il vincolo agisce sui logits come nella (2.9): si pone $z_{t,v} = -\infty$ per ogni $v \notin \mathcal{A}_t$ prima della softmax.

8.2 Implementazione

Il vincolo è realizzato da due componenti con responsabilità separate: una maschera del vocabolario (`GlossVocabularyMask`), struttura dati pura che contiene il Trie e calcola l'insieme ammesso, testabile senza modello, e un processore di logits che la applica in fase di generazione. Lo stesso calcolo dell'insieme ammesso è esposto in forma *teacher-forced*, cioè per prefissi noti in blocco, ed è consumato dagli obiettivi ausiliari del Capitolo 9: la definizione di "token ammesso" esiste in un solo punto.

8.2.1 La doppia radice

Il tokenizzatore BPE codifica lo spazio come parte del token che segue: il token per `HOUSE` e quello per `" HOUSE"` sono identificatori distinti. Il primo token di una sequenza, che non ha spazio davanti, e il primo token di ogni gloss successivo, che lo porta, appartengono quindi a insiemi diversi.

Il Trie ha perciò **due radici**, entrambe punti di ingresso all'*inizio* di un gloss:

`root` indicizza il primo token BPE di ciascun gloss codificato *senza* spazio iniziale; è usata soltanto per il primo token generato dopo il prompt;

`space_root` indicizza gli stessi primi token codificati *con* spazio iniziale (`" " + gloss`); è usata per avviare ogni gloss successivo al primo.

La regola di transizione è la seguente. All'inizio della generazione l'insieme ammesso è l'insieme dei figli di `root`. All'interno di un gloss si percorrono i figli del nodo corrente. Quando si raggiunge un nodo terminale il gloss può concludersi, e il gloss seguente deve iniziare da un figlio di `space_root`; la prosecuzione dello stesso gloss (figli del nodo terminale) resta ammessa se il gloss è prefisso di un altro.

Lo scopo è imporre il separatore agli spazi fra gloss. Con una radice unica, due gloss la cui forma superficiale è un singolo token BPE potrebbero essere emessi in successione *senza* spazio, e la loro concatenazione (per esempio `DEBUT` seguito da `RECHT`, che darebbe `DEBUTRECHT`) sarebbe una forma inesistente nel vocabolario. Poiché ogni gloss successivo al primo può iniziare solo da un token che include lo spazio, il confine di parola è esplicito nella struttura del Trie.

8.2.2 Rapporto con lo stato dell'arte

Il Trie di prefissi a livello di token non è una novità: è la struttura con cui GENRE [6] vincola la generazione ai nomi di un insieme chiuso di entità, e la stessa idea, in forma di automa, è alla base di Outlines [46] e di Synchromesh [33]. Il disallineamento fra i confini di parola del vincolo e i sotto-token del modello è noto e trattato in DOMINO [2]; l'effetto dello spazio iniziale sulla qualità dei token strutturati è misurato da Hamilton e Mimno [13]. Un lavoro recente costruisce un solo Trie a livello di carattere per insiemi finiti di stringhe [49].

La doppia radice è invece una soluzione *locale* a un caso specifico: sequenze di più elementi di un lessico chiuso, separati da spazio, con un tokenizzatore che codifica lo spazio nel token successivo. Nelle fonti consultate non ho trovato questa costruzione descritta con un nome proprio; d'altra parte l'idea (distinguere il punto d'ingresso a inizio stringa da quello dopo uno spazio) è una conseguenza diretta della tokenizzazione BPE e compare, in forma implicita, nella gestione dello spazio iniziale delle librerie di decodifica vincolata. La relazione non la presenta quindi come contributo originale, ma come adattamento necessario del Trie noto a questo tokenizzatore.

8.2.3 Numeri della costruzione

Grandezza	Valore
Gloss nel vocabolario	15 472
Nodi dei due alberi	55 123
Token distinti usati nei due alberi	4 002
Token che iniziano l'uscita (radice `root`)	1 391
Token che iniziano un gloss successivo (radice `space_root`)	1 550
Logit prodotti dal modello a ogni passo	151 936
Gloss non emettibili	0

A ogni passo il modello sceglie quindi fra poche centinaia o poche migliaia di token invece che fra tutti i 151 936 logit, e dentro un gloss la scelta si restringe ancora: dopo `DESC` restano 36 continuazioni, dopo `FLIN` una sola. Ogni gloss del vocabolario è emettibile in entrambe le posizioni, quindi nessuna delle 897 971 occorrenze di gloss del training set è irraggiungibile.

Un'ulteriore diagnostica opzionale misura la massa di probabilità rimossa dal vincolo a ogni passo; è disattivata di default e non interviene nei risultati riportati.

8.3 Il risultato controintuitivo

Ecco l'effetto del vincolo sulla cella base zero-shot.

Metrica	Senza Trie	Con Trie
ROUGE-L	0,3648	0,1373
Non-copy-token accuracy	0,0007	0,0451
Validity	0,0370	0,9055

Il vincolo fa *crollare* ROUGE-L da 0,3648 a 0,1373: una perdita apparente di 0,23. Allo stesso tempo la non-copy-token accuracy passa da 0,0007 a 0,0451, un fattore circa 64.

8.3.1 Come si spiega

Le due metriche misurano cose diverse, e il vincolo agisce su di esse in direzioni opposte.

Senza vincolo, il modello base fa la cosa per cui è stato pre-addestrato: produce inglese. Poiché il gloss è in larga parte inglese maiuscolizzato (sezione 3.2), produrre inglese ottiene un ROUGE-L rispettabile "per caso". Ma la non-copy-token accuracy, che valuta solo le posizioni dove il gloss *differisce* dal testo, è 0,0007: praticamente nulla. Il modello non ha imparato niente sul gloss; sta beneficiando della sovrapposizione lessicale del corpus.

Con il vincolo, il modello è costretto a emettere solo gloss del vocabolario. Perde la possibilità di copiare l'inglese, quindi perde il ROUGE-L gratuito. In compenso, sulle posizioni che richiedono una trasformazione reale, comincia a produrre qualcosa di corretto (0,0451 invece di 0,0007).

Detto in modo compatto: il vincolo trasforma una copia con punteggio alto in un tentativo di traduzione con punteggio basso. La metrica di sovrapposizione registra un peggioramento; la metrica che conta registra un miglioramento di quasi due ordini di grandezza.

8.3.2 L'effetto veramente importante: il vincolo garantisce la struttura

Il ROUGE-L, però, non è la ragione per cui il vincolo è presente nel sistema. La ragione è la struttura delle uscite, e va misurata con un criterio che non lasci margini: un'uscita è *strettamente valida* se *tutti* i suoi token sono gloss del vocabolario. La validity delle tabelle è più permissiva, perché accetta un'uscita con almeno metà dei token nel vocabolario (Capitolo 10), e senza vincolo le due misure divergono molto.

Cella	Validity	Stretta
Base zero-shot, senza Trie	0,0370	0,0138
Base few-shot, senza Trie	0,7604	0,1147
GRPO few-shot, senza Trie	0,9899	0,1941
Base zero-shot, con Trie	0,9055	0,9869
Base few-shot, con Trie	0,9821	0,9805
GRPO few-shot, con Trie	0,9954	0,9988
SFT, con Trie	1,0000	0,9999

Senza Trie il GRPO impara l'*aspetto* di un gloss — maiuscole, prefissi `DESC-` e `X-`, articoli cancellati — e la validity permissiva sale a 0,99. Ma solo il 19% delle uscite è fatto interamente di gloss del vocabolario: il resto contiene parole plausibili che nel vocabolario chiuso non esistono (`MULTI` dove il corpus ha `MULTUS`). Con il Trie la validità stretta è fra il 98% e il 100%, qualunque sia la policy, e il residuo ha una sola causa: il limite di 128 token. Un'uscita che

entra in un ciclo e non si chiude viene troncata, spesso a metà di un gloss, e l'ultimo pezzo non è un gloss. Su 40 000 uscite vincolate nessun token fuori vocabolario compare in una posizione diversa dall'ultima, tutte le uscite non valide sono lunghe esattamente 128 token, e nessuna è vuota.

Il componente simbolico non migliora le metriche: garantisce il vincolo che il reinforcement learning da solo non impara. È il risultato neuro-simbolico di questo lavoro. Il GRPO avvicina la forma, il Trie impone il vocabolario.

Una cosa che il vincolo *non* fa, contrariamente a quanto si potrebbe attendere, è creare il segnale di apprendimento. I gruppi a varianza nulla sono meno del 6% sia con sia senza Trie (sezione 6.4): con sette componenti di reward continue i candidati di un gruppo ricevono quasi sempre reward diverse, vincolati o no.

8.4 Il costo

Il vincolo non è gratuito. A ogni passo di generazione richiede:

- la risoluzione del nodo corrente del Trie a partire dal prefisso;
- la costruzione di una maschera booleana sulla dimensione del vocabolario;
- l'applicazione della maschera ai logits.

Il costo dominante è la costruzione della maschera, lineare in $|\mathcal{V}|$. Nel regime di questo progetto (modello piccolo, $G = 8$ candidati per gruppo, completion di al massimo 128 token) resta trascurabile rispetto al costo del passaggio in avanti della rete. Su modelli più grandi la valutazione andrebbe rifatta, e la memorizzazione delle maschere per nodo del Trie sarebbe l'ottimizzazione naturale.

8.5 Che cosa non fa il vincolo

Per evitare sovrainterpretazioni, va delimitato ciò che il Trie garantisce.

Garantisce che ogni parola emessa appartenga al vocabolario gloss e che i confini di parola siano rispettati.

Non garantisce che la *sequenza* sia grammaticalmente valida in ASL: il linguaggio riconosciuto è `gloss*`, cioè qualunque successione di gloss legali.

Non garantisce che la sequenza sia una traduzione corretta della frase di ingresso. Il vincolo è sintattico e non guarda l'ingresso.

Non impedisce le ripetizioni degeneri: `HOUSE HOUSE HOUSE` è una sequenza perfettamente ammissibile per il Trie. È il motivo per cui la reward include una componente dedicata alla ripetizione (Capitolo 7).

Il vincolo è dunque una condizione necessaria di buona formazione, non una condizione sufficiente di correttezza. Confondere le due cose porterebbe a attribuire al componente simbolico garanzie che non ha.

8.6 Da dove viene il vocabolario, e il tetto che impone

Il Trie è costruito una sola volta, all'avvio di ogni esecuzione, a partire dal vocabolario gloss. Il vocabolario è l'insieme delle parole distinte della colonna gloss del *solo training set*, senza soglie di frequenza: basta che un gloss compaia una volta. L'addestramento lo salva su file e la valutazione rilegge lo stesso file, quindi le due fasi usano lo stesso vincolo, e il test non vi contribuisce.

Ne segue un limite preciso. Un gloss che compare nel test ma mai nel training non è nel Trie, e con il vincolo attivo non può essere generato. Sulle 2 000 frasi valutate succede nel 7,15% dei casi (143 frasi, 167 occorrenze su 24 518): sono soprattutto nomi propri e parole rare, come `BRAGHETTO`, `ISLAMIFICATION`, `PARACHUTIST`. Su quelle frasi l'exact match è nullo per costruzione, per qualunque modello vincolato, quindi **il tetto dell'exact match con il Trie è** 0,9285.

Sistema	Tutte	Raggiungibili (1 857)	Irraggiungibili (143)
Regola (senza Trie)	0,5865	0,6123	0,2517
SFT	0,7662	0,8252	0
SFT con massa ammessa	0,8341	0,8983	0
SFT seguito da GRPO, zero-shot	0,8095	0,8718	0

La baseline a regole non ha questo tetto: una parola che non conosce la copia in maiuscolo, e così riproduce 89 dei 167 gloss mai visti nel training. Il confronto fra la regola e i modelli vincolati è quindi sbilanciato a sfavore dei secondi: sulle frasi che entrambi possono tradurre il vantaggio dell'SFT è di 21 punti di exact match, non di 18.

Includere nel vocabolario anche i gloss del test eliminerebbe il tetto, ma userebbe le risposte del test per decidere che cosa il modello può scrivere: è una fuga di informazione dal test, e renderebbe i numeri non difendibili. L'alternativa corretta è ammettere, per ogni frase, anche la copia in maiuscolo delle parole dell'*ingresso*, cioè lo stesso meccanismo che usa la regola. Il vincolo diventerebbe dipendente dalla frase, ma costruito solo da ciò che il modello vede.

8.7 Il costo nascosto della generazione vincolata

Il risultato della sezione 8.3 — il vincolo peggiora la metrica di sovrapposizione — non è un'anomalia locale di questo progetto. È un fenomeno riconosciuto nella letteratura sulla generazione strutturata, e conviene collocarvelo perché ne chiarisce la natura.

Il punto teorico è quello anticipato a proposito della (2.9): mascherare i logits e rinormalizzare produce una distribuzione che *non* è la distribuzione condizionata del modello all'insieme delle sequenze valide. La rinormalizzazione è locale, passo per passo, mentre il condizionamento corretto richiederebbe di tenere conto della probabilità che il prefisso corrente possa essere completato in una sequenza valida. Ne segue un errore sistematico, ed esistono metodi che lo correggono restituendo stimatori asintoticamente non distorti [50].

Le conseguenze pratiche di questa distorsione sono state documentate da due angoli distinti. Da un lato, il vincolo può degradare la qualità del contenuto generato anche quando garantisce la forma, con un costo che è stato caratterizzato come dipendente dal condizionamento sulla bozza [37]. Dall'altro, il vincolo può indurre il modello a proseguire lungo una struttura formalmente valida ma sostanzialmente sbagliata, un effetto descritto come tassa di allineamento della decodifica vincolata [55].

Nel presente lavoro la manifestazione è precisamente questa: il modello base vincolato produce sequenze di gloss ben formate (validity 0,9055) ma con punteggio di sovrapposizione

basso (0,1373), mentre senza vincolo produce sequenze mal formate (validity 0,0370) con punteggio più alto (0,3648). La differenza rispetto alla letteratura citata è nella conclusione operativa: qui il vincolo va mantenuto *nonostante* il costo, perché è ciò che rende non nullo il gradiente del reinforcement learning (sezione 6.4). Il costo sulla metrica di sovrapposizione è il prezzo pagato per avere un segnale di apprendimento.

8.8 Evidenza ablativa

L'effetto del Trie è verificato da confronti indipendenti. Sul modello base il vincolo porta la validity da 0,0370 a 0,9055 in zero-shot, e con il prompting a esempi il ROUGE-L sale da 0,1373 a 0,4664 (validity 0,9821). Senza Trie il BLEU corpus collassa a 0,0008, segno che le uscite non sono gloss. Infine l'ablazione `no-grammar` dell'addestramento GRPO, che rimuove il vincolo, raggiunge ROUGE-L 0,7590 ed exact match 0,0604, contro 0,9681 e 0,7662 dell'SFT vincolato.

Una variante distinta, l'iniezione di un blocco di glossario nel prompt di addestramento GRPO, è stata provata come ablazione separata; non fa parte del meccanismo di decodifica e non viene discussa oltre.

Capitolo 9

Obiettivi ausiliari

Il Capitolo 6 ha mostrato che il reinforcement learning su questo compito è schiacciato fra gruppi morti e gruppi a varianza nulla. Una risposta possibile è cercare segnale altrove: non in una reward su sequenze generate, ma in obiettivi ausiliari calcolabili in modalità *teacher-forced*, cioè sui riferimenti noti.

Sono stati implementati due obiettivi, entrambi aggiunti alla loss dell'SFT. Tutte le celle a confronto usano gli stessi seed e la stessa configurazione di base, e differiscono soltanto per il termine ausiliario: il confronto con l'SFT semplice è quindi controllato.

9.1 Obiettivo 1: la massa ammessa

9.1.1 Formulazione

L'idea è di premiare il modello per la probabilità che assegna *all'insieme* dei token legali, invece che al singolo token corretto. È esattamente la formulazione (2.6) introdotta nei fondamenti:

$$\mathcal{L}_{\mathrm{AM}} \;=\; -\log \sum_{v \in \mathcal{A}_t} \pi_\theta(v \mid x, y_{<t}), \tag{9.1}$$

dove $\mathcal{A}_t$ è fornito dal Trie del Capitolo 8 tramite il metodo `allowed_mask_for_prefixes` e i prefissi $y_{<t}$ sono quelli del riferimento.

Questo è apprendimento a etichette parziali [5], equivalente alla massimizzazione della verosimiglianza marginale [12]. Va segnalato che nel caso profondo la formulazione ingenua della (9.1) è, contrariamente all'intuizione, un punto di partenza competitivo e non un ripiego [39]: è la ragione per cui non è stata introdotta alcuna variante più elaborata. La loss totale diventa

$$\mathcal{L} \;=\; \mathcal{L}_{\mathrm{SFT}} \;+\; \lambda\, \mathcal{L}_{\mathrm{AM}} \tag{9.2}$$

con λ configurabile.

9.1.2 Che cosa questo obiettivo può e non può fare

Qui va ripetuta con forza l'osservazione della sezione sui fondamenti: la (9.1) è **piatta** sull'insieme ammesso. Qualunque ridistribuzione di massa *interna* ad $\mathcal{A}_t$ lascia la loss invariata.

Ne segue una previsione precisa, che è anche il criterio con cui l'obiettivo va valutato:

- l'obiettivo può migliorare la **calibrazione**: quanta probabilità il modello colloca su token legali, e quindi quanta massa il vincolo di decodifica deve rimuovere;

- l'obiettivo **non può**, da solo, migliorare l'exact match, perché non fornisce alcuna preferenza fra i token legali.

9.1.3 Un dettaglio sul gradiente, non catturato dalla previsione

La previsione appena dichiarata guarda al *valore* della (9.1), che è piatto sull'insieme ammesso. Il suo *gradiente* non lo è, e la differenza è rilevante per interpretare correttamente il risultato del Capitolo 11.

Per un token ammesso i con logit z_i e probabilità $p_i = \pi_\theta(i \mid x, y_{<t})$, derivando la (9.1) rispetto a z_i si ottiene

$$\frac{\partial \mathcal{L}_{\mathrm{AM}}}{\partial z_i} \;=\; -\,p_i\,\frac{1 - P_{\mathcal{A}}}{P_{\mathcal{A}}}, \qquad P_{\mathcal{A}} \;=\; \sum_{v \in \mathcal{A}_t} \pi_\theta(v \mid x, y_{<t}). \tag{9.3}$$

Il segno è negativo (il gradiente della loss spinge z_i verso l'alto, come atteso: la loss va minimizzata), e il fattore moltiplicativo è p_i stesso. Ogni token ammesso sale quindi in proporzione a *quanta probabilità ha già*: i token già preferiti guadagnano più degli altri, e la differenza $z_i - z_j$ fra due token ammessi si muove nella direzione di $p_i - p_j$. La distribuzione *dentro* l'insieme ammesso si affila, anche dopo la rinormalizzazione operata dal Trie in decodifica — ma l'ordinamento dei candidati non cambia, ed è per questo che la previsione resta corretta nella sua parte qualitativa ("non fornisce alcuna preferenza fra i token legali").

Un modello più affilato è più robusto al campionamento, e un guadagno di exact match sotto campionamento sarebbe compatibile con questo meccanismo pur senza contraddire la piattezza del valore della loss. La sezione 9.5 mostra tuttavia che il guadagno persiste quasi identico in decodifica greedy, dove l'affilatura da sola non dovrebbe contare: la previsione va quindi corretta, non solo completata.

9.2 Obiettivo 2: la loss strutturata

9.2.1 Formulazione

Il secondo obiettivo introduce un termine che modella le *transizioni* fra gloss consecutivi, nello spirito dei campi aleatori condizionali (*CRF*) e delle loro varianti applicate a compiti di etichettatura strutturata [10, 17, 54].

Il modello è un CRF *sparso a lunghezza esatta*: si assume che la sequenza di gloss abbia la stessa lunghezza della sequenza sorgente (che è il caso nel 27,5% del corpus, sezione 3.3) e si assegna un punteggio alle transizioni fra stati consecutivi.

Lo spazio degli stati è ridotto a 513 elementi: i 512 gloss più frequenti più uno stato aggregato `OTHER` che raccoglie tutti gli altri. Le transizioni sono stimate esclusivamente dal training set *dopo* la suddivisione, per evitare contaminazione.

Il termine è calcolato con accumulazione sparsa in precisione a 32 bit, evitando il tensore di punteggi di forma $[B, T, |\mathcal{V}|]$ che un CRF ingenuo richiederebbe.

9.2.2 Qualificazione: quanto informa realmente il gloss precedente?

Prima di spendere tempo di GPU su questo obiettivo, è stata posta la domanda diretta: *sapere il gloss precedente aggiunge informazione, dato che si conosce già il token sorgente?*

La domanda è formulabile in modo esatto con l'entropia condizionale (2.16). I conteggi sono stimati sul solo training set e la media è calcolata sui 24 324 token allineati del test, con una strategia di *backoff* a tre livelli

$$(\texttt{prev}, \texttt{src}) \;\to\; \texttt{src} \;\to\; \text{unigramma} \tag{9.4}$$

per gestire le combinazioni non osservate. Il calcolo vive in `src/analysis/transition_entropy.py` ed è eseguibile da terminale, senza GPU.

Quantità	Valore
$H(\text{gloss} \mid \text{src})$	0,6736 bit
$H(\text{gloss} \mid \text{src}, \text{prev})$	0,6277 bit
Guadagno	+0,0459 bit
Frazione dell'incertezza residua rimossa	6,8%

Il gloss precedente aggiunge circa 0,046 bit, cioè rimuove qualche punto percentuale dell'incertezza che resta dopo aver visto il token sorgente. Non è zero: esiste un prior strutturale reale. Ma è piccolo, e fissa un limite superiore realistico a quanto un termine di transizione possa contribuire.

Delle tre quantità della tabella la sola robusta è il *guadagno*: è una differenza fra due stime che condividono lo stesso backoff, quindi gran parte dell'errore di stima si cancella. I due livelli assoluti dipendono invece dalla politica di smoothing adottata per i contesti non osservati, e non vanno citati come costanti del corpus.

9.2.3 Il caveat, dichiarato

La stima soffre di una limitazione che va esposta e non nascosta: è calcolata solo sulle coppie *allineate*, cioè sul 27,5% del corpus in cui testo e gloss hanno la stessa lunghezza.

Questo è problematico in modo specifico. Il fenomeno linguistico più rilevante identificato nell'analisi dell'SFT (sezione 5.5) è la cancellazione contestuale della copula `BE`. Ma una cancellazione *cambia la lunghezza*: quei casi stanno per costruzione fuori dal sottoinsieme allineato.

Il valore +0,046 bit è dunque una stima ottenuta dove il fenomeno di interesse non c'è. Potrebbe sottostimare o sovrastimare il vero contributo. È una limitazione del metodo di stima, non un difetto dell'implementazione, e la sua risoluzione richiederebbe un allineamento non monotono fra le due sequenze.

9.3 Il contratto di non regressione

Aggiungere termini alla loss è una modifica invasiva, e il progetto adotta quindi un contratto verificato da test. Con peso nullo il percorso di addestramento è bit-identico a quello dell'SFT semplice, così che le celle che non usano gli obiettivi ausiliari non ne siano influenzate. Le opzioni che distruggono la corrispondenza fra posizione nel tensore e posizione nella sequenza (*packing*, *padding-free*) sono rifiutate. Il grafo delle transizioni è stimato solo sul training con un taglio sparso: un esempio di valutazione può usare una transizione assente, per la quale il punteggio del percorso gold vale $-\infty$. Queste righe sono escluse dalla media e contate a parte: nessuna riga è valutata contro un bersaglio che il modello non può rappresentare.

9.4 Il controllo con transizioni mescolate

Per stabilire se il guadagno della loss strutturata derivi dall'informazione sulle transizioni o dalla sola capacità aggiuntiva della testa, si confronta l'addestramento con le transizioni *vere* con quello con transizioni *mescolate*. "Mescolate casualmente" non è però una specifica sufficiente. Permutare le *destinazioni* degli archi ($a \to b$ diventa $a \to \pi(b)$) produce un grafo in cui i percorsi gold reali non esistono più: il termine strutturato non è definito su di essi

e, con l'esclusione delle righe non rappresentabili, si spegne, riducendo il controllo a un SFT puro.

Il controllo corretto conserva il *supporto* del grafo, cioè l'insieme esatto delle coppie (a, b), e permuta solo i pesi, separatamente per ogni stato di partenza. Ogni stato mantiene gli stessi successori ammessi, la stessa normalizzazione e lo stesso multinsieme di probabilità uscenti (quindi la stessa entropia); l'unica informazione distrutta è *quale* successore è probabile. I percorsi gold restano valutabili per costruzione, e il termine strutturato è attivo con la stessa frequenza del grafo vero. Il supporto stesso viene dai dati reali: il controllo isola quindi il valore dei *pesi* delle transizioni, non quello del supporto.

9.5 Risultati

Valutazione su 2 000 prompt, campionamento a $\tau = 0{,}7$ con $N = 5$ generazioni per prompt, salvo dove indicato.

Cella	ROUGE-L	Exact match	Non-copy acc.	Validity
SFT	0,9681	0,7662	0,9510	1,0000
SFT (greedy)	0,9696	0,7775	0,9547	1,0000
SFT + massa ammessa	0,9769	0,8341	0,9685	1,0000
SFT + massa ammessa (greedy)	0,9774	0,8420	0,9699	1,0000
SFT + strutturata (grafo vero)	0,9430	0,6053	0,9067	1,0000
SFT + strutturata (controllo mescolato)	0,9464	0,6078	0,9089	1,0000

La massa ammessa migliora l'SFT semplice, con exact match da 0,7662 a 0,8341. La previsione basata sul valore piatto della loss non lo avrebbe anticipato, e nemmeno l'affilatura del gradiente della sezione 9.1.3 la spiega da sola: il guadagno persiste in decodifica greedy (0,7775 $\rightarrow$ 0,8420), dove il rumore di temperatura non conta. Il meccanismo esatto resta quindi non spiegato dalla sola (9.3).

La loss strutturata invece *non* migliora: exact match 0,6053 contro 0,7662 dell'SFT. Soprattutto, **fallisce il proprio controllo negativo**: il grafo mescolato, che ha lo stesso supporto ma pesi privi di significato, ottiene 0,9464/0,6078, cioè marginalmente *sopra* il grafo vero. L'effetto dell'obiettivo strutturato non è quindi attribuibile alla struttura delle transizioni, coerentemente con il guadagno informativo di soli 0,046 bit stimato nella sezione 9.2.2. Un obiettivo ausiliario va promosso non perché è implementato e i test passano, ma perché una previsione derivata dalla sua forma funzionale è verificata da un controllo: qui è vero per la massa ammessa e falso per la loss strutturata.

Capitolo 10

Metriche di valutazione

Le metriche non sono uno strumento neutrale di misura: sono parte del disegno sperimentale, e su un corpus saturo come quello descritto nel Capitolo 3 la scelta della metrica determina la conclusione. Questo capitolo documenta i loro difetti misurati e le metriche introdotte per rimediarvi.

10.1 Il difetto di ROUGE-L, misurato

L'implementazione di ROUGE-L impiegata (che segue la convenzione più diffusa nella letteratura di riferimento, così da restare confrontabile con i numeri pubblicati) esegue due normalizzazioni prima del confronto:

1. porta tutto in minuscolo (*case-insensitive*);
2. spezza i token sui caratteri non alfanumerici.

Su un compito generico sono scelte innocue. Su un compito in cui il gloss è distinto dal testo *proprio* dal maiuscolo e dai prefissi con trattino, sono devastanti. Ecco le misure dirette.

10.1.1 Il maiuscolo non conta

Generato	Riferimento	ROUGE-L
`cat sat`	`CAT SAT`	1,00
`the cat sat`	`CAT SAT`	0,80

La prima riga è il problema in forma pura: un'uscita in minuscolo, cioè inglese non convertito in gloss, ottiene il punteggio massimo. La seconda mostra che anche mantenendo un articolo che il gloss elimina si resta a 0,80.

Poiché il modello base, non addestrato, produce esattamente inglese in minuscolo, ne segue che ROUGE-L assegna un punteggio alto a un sistema che non ha eseguito *nessuna* conversione. È la spiegazione del risultato apparentemente paradossale della sezione 8.3: il modello senza vincolo ottiene 0,3648 facendo la cosa sbagliata.

10.1.2 I prefissi grammaticali vengono distrutti

La tokenizzazione sui caratteri non alfanumerici spezza i marcatori del gloss:

Stringa	Token prodotti
`DESC-GOOD X-WE`	`['desc', 'good', 'x', 'we']`

Il prefisso `DESC-`, che porta informazione grammaticale, diventa un token indipendente. La conseguenza è che due gloss diversi condividono metà dei loro token:

Generato	Riferimento	ROUGE-L
`DESC-GOOD`	`DESC-BAD`	0,50
`X-WE`	`X-I`	0,50

Confondere *buono* con *cattivo* costa metà del punteggio. Confondere *noi* con *io* costa metà del punteggio. Un sistema che indovina il prefisso e sbaglia sistematicamente la parola ottiene 0,50 su ogni gloss marcato.

10.1.3 La conclusione operativa

ROUGE-L, su questo corpus e con questa implementazione, è una metrica di *copia*: premia la sovrapposizione superficiale di caratteri alfanumerici in minuscolo, che è precisamente la relazione che il corpus sintetico garantisce per costruzione (sezione 3.2).

Non è stata rimossa, per due ragioni: è l'unica metrica su cui esistano numeri pubblicati confrontabili, e la sua degenerazione è essa stessa un risultato da documentare. È stata invece È stata invece **degradata** da metrica primaria a metrica di confronto, insieme alle altre metriche di sovrapposizione (BLEU corpus, chrF, gloss-F1), anch'esse saturate dalla copia.

10.2 Le metriche primarie

Al posto di ROUGE-L, le metriche promosse a primarie sono due.

10.2.1 Exact match normalizzato

Confronto di uguaglianza dopo normalizzazione degli spazi. È binario, severo e non manipolabile: non esiste una strategia che aumenti l'exact match senza produrre l'uscita corretta.

Il suo pregio principale su questo corpus è che *ha ancora risoluzione*. Fra baseline a regole e SFT (Capitolo 5) ROUGE-L è praticamente indistinguibile (0,9685 contro 0,9681), mentre l'exact match separa nettamente i due sistemi (0,5865 contro 0,7662).

10.2.2 Non-copy-token accuracy

È la metrica progettata specificamente per questo corpus, e il contributo metodologico principale del capitolo.

L'idea: si considerano soltanto le posizioni in cui il gloss di riferimento *differisce* dal token sorgente maiuscolizzato. Su quelle posizioni, e solo su quelle, si misura l'accuratezza. Formalmente, detto $\mathcal{N} = \{t : r_t \neq \mathrm{upper}(x_t)\}$ l'insieme delle posizioni non banali,

$$\texttt{non_copy_token_accuracy} = \frac{|\{t \in \mathcal{N} : g_t = r_t\}|}{|\mathcal{N}|}. \tag{10.1}$$

Sui 2 000 prompt di valutazione, $|\mathcal{N}| = 9\,350$ token: un campione ampio, non un residuo marginale.

Il potere discriminante è evidente:

Sistema	ROUGE-L	Non-copy accuracy
Base zero-shot senza Trie	0,3648	0,0007
Base zero-shot con Trie	0,1373	0,0451
GRPO da modello base (few-shot)	0,6082	0,4692
Baseline a regole	0,9685	0,9424
SFT	0,9681	0,9510

La colonna di destra ordina i sistemi in modo interpretabile e, soprattutto, assegna al modello base senza vincolo il valore che merita: 0,0007, cioè niente. La colonna di sinistra gli assegna 0,3648, che è più del doppio di quanto ottenga lo stesso modello con il vincolo di decodifica attivo.

10.3 Validity

La *validity* misura la frazione di uscite ben formate.

Un'uscita è dichiarata **non valida** se si verifica almeno una delle condizioni:

1. meno del 50% dei suoi token appartiene al vocabolario gloss;
2. ha più di 4 token e il rapporto fra token distinti e token totali (*unicità*) è inferiore a 0,3.

La prima condizione cattura le uscite che non sono gloss. La seconda cattura le ripetizioni degeneri del tipo `HOUSE HOUSE HOUSE HOUSE HOUSE`, che soddisfano la prima condizione perfettamente (tutti i token sono in vocabolario) e sono comunque prive di valore. La soglia di 4 token evita di penalizzare uscite legittimamente brevi in cui una ripetizione può essere reale.

10.4 Pass@k

Pass@k segue la media empirica (2.18), con soglia di correttezza posta a ROUGE-L $\geq 0{,}3$; non si impiega la forma combinatoria non distorta. La soglia va dichiarata perché il valore dipende interamente da essa, e 0,3 è su questo corpus *permissiva*: dato il difetto della sezione 10.1, un'uscita che la supera non è necessariamente una traduzione utile. Pass@k va quindi letta come misura di *copertura* del campionamento, non di qualità.

10.5 BLEU, chrF e gloss-F1

BLEU è calcolato a livello di corpus (BLEU-c) e chrF come F-score su n-grammi di caratteri; entrambi seguono l'implementazione `sacrebleu`. Il gloss-F1 è l'F-score sull'insieme dei gloss emessi rispetto al riferimento. Il gloss ASL ha la punteggiatura come token autonomo per costruzione del corpus (`X-Y WILL DESC-NOT FLINCH .`), quindi il dato è legittimamente pretokenizzato. I confronti **fra** le celle di questo lavoro restano validi, perché il protocollo è identico per tutte; i valori assoluti di BLEU e chrF **non** sono confrontabili con numeri pubblicati su dato detokenizzato.

10.6 Incertezza: due fonti, nessuna delle due quantificata

Le tabelle del capitolo sui risultati riportano valori puntuali, senza intervalli. È una scelta, e va motivata, perché le fonti di incertezza sono due e hanno statuto molto diverso.

La prima è la **varianza di campionamento sul test set**: i 2 000 prompt sono un campione, e su un campione diverso i numeri sarebbero leggermente diversi. È quantificabile per bootstrap, con una precisazione necessaria: il ricampionamento andrebbe **clusterizzato per prompt**. La valutazione genera infatti $N = 5$ uscite per prompt, e le cinque uscite dello stesso prompt non sono osservazioni indipendenti — condividono la frase di ingresso, la sua difficoltà e il suo riferimento. Ricampionare le singole uscite come se fossero indipendenti gonfierebbe la dimensione effettiva del campione di un fattore fino a cinque, restituendo intervalli artificialmente stretti. Il bootstrap clusterizzato ricampiona i *prompt* con reimmissione, portandosi dietro tutte le uscite di ciascun prompt selezionato.

La seconda è la **variabilità fra semi**, ed è quella che conta davvero per confrontare due celle. Questa relazione **non la quantifica**: a seme fissato l'addestramento è deterministico bit per bit (sezione 11.10), quindi ripetere una cella non produce alcuna dispersione, e nessuna cella è stata replicata con semi diversi.

Riportare solo la prima darebbe un'impressione di precisione fuorviante: l'intervallo di campionamento su 2 000 prompt è stretto, ma non dice nulla su quanto un risultato si riprodurrebbe cambiando inizializzazione. Le conclusioni del lavoro sono state perciò formulate su divari abbastanza ampi da non dipendere da questa distinzione — il divario fra reinforcement learning e baseline a regole è di ordine 0,35 di ROUGE-L, quello fra massa ammessa e SFT semplice di quasi sette punti di exact match — e nessuna conclusione è tratta da differenze dell'ordine del millesimo.

10.7 Il protocollo di valutazione, riassunto

Perché i numeri del capitolo successivo siano interpretabili, il protocollo è fissato e identico per tutte le celle:

Parametro	Valore
Prompt valutati	2 000 (dal test set)
Generazioni per prompt	$N = 5$
Strategia di decodifica	campionamento, $\tau = 0{,}7$
Incertezza riportata	nessuna (si veda sopra)
Riferimento obbligatorio	baseline a regole

Il campionamento a temperatura 0,7 invece del greedy è una scelta che va motivata: consente di stimare pass@k e di misurare la varianza delle uscite, che è l'informazione necessaria per interpretare il supporto dei rollout (sezione 6.4). Il costo è che i numeri di exact match sono leggermente inferiori a quelli che si otterrebbero in greedy; poiché il confronto fra celle usa lo stesso protocollo, i confronti restano validi.

Capitolo 11

Risultati

Tutti i numeri di questo capitolo seguono il protocollo fissato nel Capitolo 10: 2 000 prompt del test set, $N = 5$ generazioni per prompt, campionamento a temperatura 0,7, metriche versione 2.

11.1 Provenienza dei numeri

Ogni riga delle tabelle che seguono è letta da *un solo* file `eval_final.json` prodotto dalla pipeline di valutazione, scegliendo per ciascuna cella il run più recente conservato. Tutti i file dichiarano `metrics_version = 2`; nessun valore proviene da documenti derivati o da ricalcoli successivi. Le esecuzioni prodotte prima delle revisioni del codice descritte in questa relazione non compaiono qui: sono prive di un identificativo di run e di un riferimento al config che le ha generate, quindi non sono attribuibili a nessuna cella sperimentale.

Una cella progettata è stata eliminata dal disegno. Doveva isolare l'effetto della revisione dello stack di reward mantenendo i pesi storici, ma il suo config assegnava ai componenti esattamente gli stessi pesi della cella GRPO di riferimento: l'unica differenza era l'azzeramento di un componente già inattivo per default. A parità di seme le due celle producono generazioni identiche byte per byte, cioè sono la stessa esecuzione sotto due nomi. Lo stack storico *è* la configurazione di reward predefinita, quindi non esiste un'ablazione che lo isoli e la cella è stata rimossa dal progetto anziché riportata come evidenza indipendente.

11.2 La tabella principale

Cella	ROUGE-L	EM	BLEU-c	GlossF1	Validity	non-copy
Modello non addestrato						
Base zero-shot senza Trie	0,3648	0,0007	0,0008	0,2279	0,0370	0,0007
Base zero-shot con Trie	0,1373	0,0017	0,0270	0,1186	0,9055	0,0451
Base few-shot con Trie	0,4664	0,0148	0,1880	0,4026	0,9821	0,3555
Reinforcement learning dal modello base						
GRPO zero-shot	0,5173	0,0078	0,2322	0,5593	0,9954	0,1927
GRPO few-shot	0,6082	0,0544	0,3419	0,6220	0,9954	0,4692
Supervisione						
SFT	0,9681	0,7662	0,9316	0,9698	1,0000	0,9510
SFT, decodifica greedy	0,9696	0,7775	0,9362	0,9721	1,0000	0,9547
SFT seguito da GRPO, few-shot	0,9628	0,7296	0,9220	0,9648	1,0000	0,9471
SFT seguito da GRPO, zero-shot	0,9744	0,8095	0,9443	0,9757	1,0000	0,9650
Obiettivi ausiliari in fase SFT						
Massa ammessa	0,9769	0,8341	0,9518	0,9783	1,0000	0,9685
Massa ammessa, greedy	**0,9774**	**0,8420**	0,9533	0,9791	1,0000	0,9699
Loss strutturata, grafo vero	0,9430	0,6053	0,8803	0,9466	1,0000	0,9067
Loss strutturata, controllo	0,9464	0,6078	0,8863	0,9474	1,0000	0,9089
Baseline a regole	**0,9685**	0,5865	0,8907	0,9665	—	0,9424
uppercase(text)	0,6454	—	—	—	—	—

Le ablazioni sono raccolte a parte, perché isolano un fattore ciascuna invece di proporre un metodo:

Ablazione	ROUGE-L	EM	BLEU-c	GlossF1	non-copy
GRPO senza Trie (`no-grammar`)	0,7590	0,0604	0,4435	0,7175	0,4730
SFT seguito da GRPO, rollout a temperatura 1,3	0,9634	0,7419	0,9185	0,9641	0,9500
Denominatore Dr. GRPO	0,5373	0,0360	0,2672	0,5246	0,4043
Penalità KL ridotta	0,9644	0,7414	0,9244	0,9661	0,9499
Reward di sola validità dell'edit	0,6373	0,0608	0,3684	0,6432	0,4460
Stack di reward ridotto	0,9634	0,7346	0,9228	0,9653	0,9488
Glossario nel prompt, zero-shot	0,5159	0,0098	0,2317	0,5605	0,1858
Glossario nel prompt, few-shot	0,6094	0,0544	0,3419	0,6261	0,4636

La riga in grassetto della prima tabella è il riferimento che rende leggibile il resto: una trasformazione a regole, non addestrata, che nessuna cella supera in modo convincente sulle metriche di sovrapposizione.

Due avvertenze sulla lettura. La colonna BLEU riporta il valore *a corpus*, calcolato su dato pre-tokenizzato: il gloss ha la punteggiatura come token separato (`X-Y WILL DESC-NOT FLINCH .`). I confronti *fra* celle restano validi, perché il protocollo è identico per tutte; i valori assoluti **non** sono confrontabili con numeri pubblicati su dato detokenizzato. Inoltre, dove non altrimenti indicato, la decodifica è campionata a temperatura 0,7 con $N = 5$ generazioni per prompt; le due righe marcate *greedy* riportano la stessa cella decodificata in modo deterministico, ed è la modalità in cui i valori sono più alti perché non c'è varianza di campionamento da assorbire.

11.3 Osservazione 1: nessun risultato con RL supera la regola

Il miglior risultato ottenuto con reinforcement learning a partire dal modello base è 0,6082 di ROUGE-L (GRPO few-shot). La baseline a regole ottiene 0,9685. La trasformazione *metti tutto in maiuscolo* ottiene 0,6454.

Cioè: il miglior risultato con RL dal modello base è **inferiore** a quello ottenibile maiuscolizzando l'input, e distante 0,36 da una tabella di corrispondenze costruita contando parole nel training set.

Su exact match il divario è più netto: 0,0544 contro 0,5865, un fattore dieci. Su gloss F1: 0,6220 contro 0,9665.

Va detto con precisione che cosa questo significa e che cosa non significa. Il GRPO dal modello base *apprende*: partendo da 0,4664 della stessa cella non addestrata in few-shot, arriva a 0,6082, e la validity sale a 0,9954. Non è che l'ottimizzazione non produca nulla. È che il livello raggiunto resta lontano da quello di una trasformazione a regole di poche righe, e la domanda residua non è "quanto migliora" ma "perché non può bastare".

11.4 Osservazione 2: l'SFT è al tetto del corpus

L'SFT ottiene ROUGE-L 0,9681, sostanzialmente pari alla baseline a regole (0,9685) e superiore al miglior valore pubblicato in letteratura su ASLG-PC12 (0,9587, [32]).

Un modello da 0,5 miliardi di parametri, con adattatori LoRA, addestrato per poco più di due epoche su una singola GPU, che eguaglia lo stato dell'arte pubblicato e insieme una tabella di corrispondenze non addestrata. La lettura corretta non è che il sistema sia migliore: è che la metrica ha esaurito la propria capacità discriminante.

Il confronto corretto fra SFT e regola non è dunque su ROUGE-L, dove i due sono indistinguibili ($-0{,}0004$ a favore della regola; con decodifica greedy l'SFT passa a 0,9696, cioè $+0{,}0011$), ma su exact match, dove l'SFT ottiene 0,7662 contro 0,5865: quasi diciotto punti, che corrispondono a una riduzione del tasso di errore dal 41% al 23%. Ciò che l'SFT aggiunge alla regola non è visibile alle metriche di sovrapposizione, ed è precisamente per questo che la baseline va stampata accanto a ogni numero.

Il contrasto con PHOENIX-2014T rende la cosa evidente. Su quel corpus, dove la sovrapposizione lessicale fra gloss e testo è molto inferiore (sezione 3.2), i migliori risultati T2G riportati sono intorno a ROUGE-L 55,18 [28]. Circa quaranta punti in meno, sullo stesso compito nominale e con metodi comparabili. La differenza non è nei metodi: è nel dato.

11.5 Osservazione 3: il vincolo simbolico e le due metriche

Le prime righe della tabella principale, lette insieme alla non-copy-token accuracy, sono il risultato neuro-simbolico del lavoro.

Cella	ROUGE-L	Validity	Non-copy accuracy
Base zero-shot senza Trie	0,3648	0,0370	0,0007
Base zero-shot con Trie	0,1373	0,9055	0,0451

Il vincolo abbassa ROUGE-L di 0,23, alza la validity di un fattore 24 (da 0,0370 a 0,9055) e alza la non-copy accuracy di un fattore circa 64. Il meccanismo è quello discusso nel Capitolo 8: senza vincolo il modello produce inglese in minuscolo, che la metrica premia per il difetto documentato nella sezione 10.1; con il vincolo produce gloss, male, ma producendo gloss.

La validity di 0,0370 nella prima riga è il dato che chiude la questione: il 96% delle uscite del modello senza vincolo non è nemmeno una sequenza di gloss ben formata. Assegnare loro ROUGE-L 0,3648 è un artefatto della metrica, non una misura di qualità.

La conferma su una cella addestrata viene dall'ablazione `no-grammar`, che ripete l'addestramento GRPO senza vincolo di decodifica: ROUGE-L 0,7590 ed exact match 0,0604, contro 0,9681 e 0,7662 dell'SFT vincolata. Il valore di ROUGE-L, isolato, sembrerebbe promettente; l'exact match mostra che quasi nessuna uscita è la sequenza di gloss corretta. Sulla struttura il quadro è lo stesso: la validity permissiva della cella arriva a 0,9899, ma le uscite fatte interamente di gloss del vocabolario sono solo il 19,4% (Capitolo 8). Senza Trie il GRPO impara l'aspetto di un gloss, non il vocabolario chiuso.

11.5.1 Una conferma indipendente dalla scelta della metrica

Su questa cella esiste una verifica che non dipende dalla non-copy accuracy né da alcuna metrica introdotta da questo lavoro, e che è quindi la prova più difficile da contestare. Sulle *stesse* generazioni, senza vincolo:

Metrica	Valore	Sensibile al maiuscolo?
ROUGE-L	0,3648	no
chrF (a corpus)	1,18 su 100	sì
BLEU (a corpus)	0,0008	sì

Le tre metriche sono calcolate sullo stesso file di generazioni, nella stessa esecuzione. Le due sensibili al maiuscolo assegnano un valore prossimo a zero; quella insensibile assegna 0,36. Il rapporto è di circa un fattore 30.

Questo è il difetto della sezione 10.1 osservato su 10 000 generazioni reali invece che su casi costruiti a mano: il modello produce inglese minuscolo, ROUGE-L lo premia perché normalizza via il maiuscolo, e ogni metrica che il maiuscolo lo guarda registra il fallimento. Per confronto, sulla cella con vincolo attivo chrF sale a 13,21, cioè di un fattore 11 rispetto a 1,18 — nella direzione opposta a quella indicata da ROUGE-L.

11.6 Osservazione 4: le lunghezze non mostrano deriva verso la verbosità

Un'ipotesi corrente sul reinforcement learning è che tenda a produrre uscite più lunghe, perché la reward correla con la lunghezza [41]. Va verificata sui dati. La tabella riporta, per ciascuna cella, la lunghezza media in token della generazione, il rapporto con la lunghezza media del riferimento (12,26 token, identica per tutte le celle essendo lo stesso test set), l'errore medio firmato e l'errore medio assoluto, calcolati sulle 10 000 generazioni conservate di ciascuna cella.

Cella	Gen.	Rapporto	Errore firmato	Errore assoluto
Base zero-shot senza Trie	10,09	0,823	−2,17	6,98
Base zero-shot con Trie	13,96	1,139	+1,70	11,24
Base few-shot con Trie	13,14	1,072	+0,88	5,12
GRPO da base, zero-shot	11,42	0,931	−0,84	2,17
GRPO da base, few-shot	11,90	0,971	−0,36	2,55
SFT	12,31	1,004	+0,05	0,11
SFT seguito da GRPO, zero-shot	12,31	1,004	+0,05	0,10
SFT seguito da GRPO, few-shot	12,34	1,006	+0,08	0,16
Massa ammessa	12,30	1,004	+0,05	0,09

La lettura contraddice l'ipotesi, e in modo netto. Entrambe le celle addestrate con GRPO a partire dal modello base producono uscite *più corte* del riferimento (rapporti 0,931 e 0,971, errori firmati negativi). Se esistesse una pressione della reward verso la verbosità, agirebbe nella direzione opposta a quella osservata.

La colonna informativa è l'errore *assoluto*. Le celle che partono da un SFT hanno errore assoluto fra 0,09 e 0,16: azzeccano la lunghezza quasi sempre. Le celle GRPO dal modello base hanno 2,17 e 2,55, con errori firmati piccoli: sbagliano la lunghezza di circa due token per volta, in entrambe le direzioni, e gli errori si compensano nella media. Il modello base non addestrato, con o senza vincolo, sbaglia di sette o undici token.

Il problema delle celle con RL dal modello base non è dunque un bias di lunghezza ma una *varianza* di lunghezza. È coerente con la struttura della reward, che penalizza attivamente l'allungamento: la verbosità non può essere il canale di guadagno.

11.7 Osservazione 5: il GRPO non degrada una policy già convergente

Applicare il GRPO a una policy già addestrata con SFT non produce né un miglioramento sostanziale né il degrado che ci si potrebbe attendere da un'ottimizzazione prolungata su un surrogato imperfetto:

Cella	ROUGE-L	EM
SFT soltanto	0,9681	0,7662
SFT seguito da GRPO, zero-shot	0,9744	0,8095
SFT seguito da GRPO, few-shot	0,9628	0,7296

La cella zero-shot è sopra l'SFT di +0,0063 di ROUGE-L e +0,0433 di exact match; quella few-shot è sotto di −0,0053 e −0,0366. L'effetto è inferiore al punto percentuale in entrambe le direzioni, e le due celle non sono appaiate per modalità di prompting, quindi la differenza fra loro non è attribuibile al solo GRPO. La conclusione onesta è che, su una policy già convergente e con la penalità KL attiva ($\beta = 0{,}04$), il GRPO lascia il sistema sostanzialmente dov'era.

Il meccanismo che lo spiega è quello dei gruppi a varianza nulla discusso nel Capitolo 6, ed è misurato: nelle due celle SFT seguito da GRPO fra il 61% e il 71% dei gruppi ha varianza nulla, contro meno del 6% delle celle GRPO dal modello base (sezione 6.4). Una policy SFT convergente risolve correttamente la gran parte dei prompt, e per quei prompt gli otto candidati sono la stessa sequenza e ricevono la stessa reward: il vantaggio è esattamente zero e non c'è alcun aggiornamento. Il gradiente che resta proviene dalla minoranza di prompt su cui la policy era incerta, cioè da un sottoinsieme selezionato e atipico; la penalità KL impedisce

che l'aggiornamento su quel sottoinsieme sposti la policy lontano dal punto di partenza. Il risultato è un'oscillazione di frazioni di punto, che è esattamente ciò che si osserva.

11.8 Osservazione 6: gli obiettivi ausiliari

I due obiettivi del Capitolo 9 sono stati addestrati come varianti della cella SFT zero-shot, con un solo fattore cambiato: il termine ausiliario, di peso $\lambda = 0{,}1$ con rampa lineare sui primi 200 passi. La valutazione è la stessa della cella SFT: 2 000 prompt del test set, decodifica vincolata dal Trie, campionamento a $T = 0{,}7$ con 5 completion per prompt.

Cella	Passi (epoche)	ROUGE-L	Exact match	chrF
SFT soltanto	10 000 (2,24)	0,9681	0,7662	96,36
Massa ammessa	13 000 (2,91)	**0,9769**	**0,8341**	97,55
Loss strutturata, grafo vero	5 600 (1,25)	0,9430	0,6053	93,16
Loss strutturata, controllo mescolato	5 600 (1,25)	0,9464	0,6078	93,67

La durata dell'addestramento non è la stessa, per scelta. Ogni cella si ferma per early stopping sulla loss di validazione interna (valutata ogni 200 passi, pazienza di tre valutazioni), e quella loss è la loss *totale*: LM più λ volte il termine ausiliario. Ciascuna cella si ferma quindi quando smette di migliorare il *proprio* obiettivo, e le durate differiscono. È una scelta deliberata e non un difetto del confronto: il confronto misura ogni obiettivo nelle condizioni in cui il suo stesso criterio di arresto lo porta. Va però tenuta presente in ogni lettura dei delta qui sotto.

11.8.1 La massa ammessa

Rispetto all'SFT la cella guadagna +0,0088 di ROUGE-L e +0,0679 di exact match. Le due metriche vanno lette in modo diverso. Su ROUGE-L il guadagno è dell'ordine del millesimo, su una scala già satura in cui anche la baseline a regole non addestrata si colloca a 0,9685: non sostiene da solo alcuna conclusione. Su exact match il guadagno è di quasi sette punti, che su una metrica non satura è un effetto di entità diversa. L'argomento che segue è costruito sull'exact match.

Il risultato va confrontato con la previsione dichiarata in anticipo nel Capitolo 9: la loss è piatta *in valore* sull'insieme ammesso, quindi non esprime preferenze fra token legali; ma il suo gradiente non è piatto, e già in sede di previsione si era osservato che un effetto sull'exact match non era per questo escluso. Restano due spiegazioni alternative da escludere prima di attribuire il guadagno all'obiettivo.

1. **Più addestramento**: la cella ha fatto il 30% di passi in più dell'SFT (13 000 contro 10 000), perché il termine ausiliario ha ritardato l'early stopping.

2. **Una distribuzione più affilata sotto campionamento**. Il valore della loss è piatto rispetto a ridistribuzioni interne all'insieme ammesso, ma il suo gradiente non lo è: per un token ammesso, $\partial \mathcal{L}_{\mathrm{AM}} / \partial z_i = -\,p_i(1 - P_{\mathcal{A}})/P_{\mathcal{A}}$, dove $P_{\mathcal{A}}$ è la massa dell'insieme. Ogni logit ammesso sale quindi in proporzione alla propria probabilità: i token già probabili guadagnano più degli altri, e la distribuzione *dentro* l'insieme si affila anche dopo la rinormalizzazione del Trie. L'ordinamento dei token ammessi però non cambia, perché $z_i - z_j$ si muove nella direzione di $p_i - p_j$. Sotto campionamento a $T = 0{,}7$ un modello più affilato si allontana meno spesso dalla sua prima scelta, e l'exact match sale anche a parità di prima scelta: sarebbe un effetto di calibrazione, non di competenza.

La seconda spiegazione si verifica valutando le due celle in modo *deterministico* (greedy: si prende sempre il token più probabile, nessuna casualità). Poiché l'affilatura conserva l'ordinamento, in greedy il solo termine ausiliario non dovrebbe cambiare le risposte: se il vantaggio resta, la cella ha imparato risposte migliori; se sparisce, era calibrazione. Le stesse due celle, dagli stessi checkpoint, valutate in greedy:

Decodifica	SFT soltanto		Massa ammessa	
	ROUGE-L	Exact match	ROUGE-L	Exact match
Campionata ($T = 0{,}7$)	0,9681	0,7662	0,9769	0,8341
Greedy	0,9696	0,7775	0,9774	0,8420

Il vantaggio resta: +0,0645 di exact match in greedy contro +0,0679 in campionamento. La seconda spiegazione è esclusa, e la tabella mostra anche perché: per entrambi i modelli passare da campionamento a greedy cambia l'exact match di appena un punto. Le distribuzioni sono già così concentrate che il campionamento a $T = 0{,}7$ quasi non se ne allontana, e un'ulteriore affilatura non aveva margine per spostare il risultato.

Resta la prima spiegazione, la durata, che però non è una variabile indipendente: sono le 3 000 iterazioni in più che il termine ausiliario ha reso utili. L'SFT non poteva sfruttarle, perché la sua loss di validazione era già ferma (0,0291 a 8 000 passi, 0,0286 a 10 000). La lettura più semplice compatibile con i dati è che il termine agisca da regolarizzatore e rimandi il plateau. Non è verificabile dai checkpoint esistenti: è conservato solo il migliore di ciascuna run, per cui la cella con massa ammessa non può essere valutata a 10 000 passi.

La conclusione, con le cautele dovute: sotto questo protocollo aggiungere il termine migliora l'exact match di circa sette punti, il guadagno non è un effetto della decodifica, e la cella con massa ammessa è il miglior risultato dell'intero lavoro su ogni metrica misurata. Resta un solo seme per cella, e la distinzione fra "il termine insegna qualcosa" e "il termine consente di addestrare più a lungo" non è separabile dai dati raccolti.

11.8.2 La loss strutturata

Con il grafo vero la cella è *peggiore* dell'SFT su tutte le metriche: −0,0251 di ROUGE-L e −0,1609 di exact match. Si è anche fermata molto prima (5 600 passi, 1,25 epoche): il termine strutturato pesa circa 0,32 su una loss di validazione totale di 0,40, per cui l'arresto anticipato è governato dal plateau della testa strutturata e non da quello del modello linguistico.

Con i dati disponibili non si possono separare le due letture: modello meno addestrato, oppure termine strutturato che interferisce con l'apprendimento della parte linguistica. In entrambe le letture, in questa configurazione il risultato non fornisce alcuna evidenza a favore dell'obiettivo, ed è coerente con il margine modesto stimato in anticipo (+0,046 bit, sezione 9.2.2).

11.8.3 Il controllo mescolato: il criterio 2 non è soddisfatto

Il controllo conserva il supporto del grafo e permuta soltanto i pesi, secondo il disegno fissato nella sezione 9.4. Il confronto con il grafo vero è pulito: stessa durata (5 600 passi per entrambi) e termine strutturato attivo nel 100% dei passi registrati (1 149 su 1 149), zero righe non rappresentabili.

La testa strutturata *distingue* i due grafi: sul grafo vero la loss di validazione migliore è 0,398, su quello mescolato 0,473. Le transizioni vere sono quindi più facili da modellare. Ma questo non si trasferisce alla traduzione: il controllo ottiene ROUGE-L 0,9464 ed exact match 0,6078, contro 0,9430 e 0,6053 del grafo vero. Le differenze, 0,0034 e 0,0025, sono a favore del *controllo*.

Per il criterio fissato in anticipo il risultato è netto: se transizioni vere e mescolate danno lo stesso risultato, il contributo non viene dall'informazione strutturale e **l'obiettivo va abbandonato**. Il peggioramento rispetto all'SFT, praticamente identico con i due grafi, è un effetto del termine in sé e dell'arresto anticipato che induce, non delle transizioni. È coerente con il margine di +0,046 bit stimato prima di addestrare: un'informazione così piccola, che un decoder autoregressivo riceve comunque condizionando sul gloss precedente, non lasciava spazio a un guadagno misurabile.

Vale la pena sottolineare che questo è il risultato di un controllo negativo *previsto prima dell'esperimento*. Senza di esso, la cella strutturata sarebbe stata riportata come un obiettivo che peggiora le metriche, e la spiegazione naturale sarebbe stata l'arresto anticipato. Il controllo mostra che la spiegazione è un'altra: qualunque cosa la testa strutturata stia imparando, non è la struttura delle transizioni.

11.9 La decomposizione dell'effetto

La domanda che chiude il capitolo: quanto dell'apparente miglioramento è attribuibile al reinforcement learning?

Si consideri il percorso dal punto di partenza peggiore al miglior risultato con RL dal modello base:

Passaggio	ROUGE-L	Variazione
Base zero-shot con Trie	0,1373	—
Base few-shot con Trie	0,4664	**+0,33** (prompting, nessun addestramento)
GRPO da modello base, few-shot	0,6082	+0,14 (RL)

Il totale è circa +0,47, di cui circa 0,33 — il 70% — viene dal solo prompting, cioè dall'inserimento di tre esempi recuperati nel prompt: nessun parametro aggiornato, nessun tempo di GPU per addestramento. I restanti 0,14 sono attribuibili al reinforcement learning a parità di prompting, poiché entrambe le righe sono few-shot.

La conseguenza per la lettura di questa relazione è precisa: **il fattore sperimentale più efficace misurato in tutto il progetto è il prompting**, e non il metodo di addestramento. Se si dovesse riprogettare l'esperimento, il fattore primario sarebbe la griglia prompting × decodifica, con il metodo di addestramento come fattore secondario applicato *dopo* aver identificato la combinazione di prompting e decodifica più favorevole.

11.10 Determinismo delle esecuzioni

Una domanda naturale davanti a tabelle con differenze di pochi millesimi è quanto di quelle differenze sia rumore di addestramento. Su questa pipeline la risposta è insolitamente semplice: **a seme fissato l'addestramento è deterministico bit per bit**. Due esecuzioni della stessa cella, lanciate indipendentemente con la stessa configurazione e lo stesso seme, hanno prodotto 10 000 generazioni identiche byte per byte e quindi metriche identiche a tutte le cifre.

Questo ha due conseguenze opposte, ed entrambe vanno dichiarate.

La prima è favorevole: ogni differenza osservata fra due celle che condividono seme e configurazione, e differiscono per un solo fattore, è interamente attribuibile a quel fattore. Non c'è una componente di rumore da sottrarre. È il motivo per cui il confronto fra la loss strutturata e il suo controllo mescolato, che differiscono soltanto per la permutazione dei pesi del grafo, è interpretabile pur essendo di pochi millesimi.

La seconda è restrittiva: poiché nessuna cella è stata replicata con semi diversi, **non esiste in questo lavoro alcuna stima della variabilità fra semi**. Il determinismo garantisce che i numeri siano riproducibili, non che siano robusti a una diversa inizializzazione o a un diverso ordine dei dati. Una differenza di pochi millesimi fra due celle è dunque reale nel senso che si riprodurrà a ogni riesecuzione, ma non è per questo generalizzabile: potrebbe invertirsi cambiando seme, e nulla in questi dati permette di escluderlo.

Le conclusioni del capitolo sono state formulate tenendone conto. Il divario fra reinforcement learning e baseline a regole è di ordine 0,35 di ROUGE-L, e nessuna variabilità plausibile fra semi lo copre. Il vantaggio della massa ammessa sull'SFT è di +0,0088 di ROUGE-L e +0,0679 di exact match: il secondo è abbastanza ampio da resistere a qualunque lettura ragionevole, il primo no, ed è per questo che l'esito dell'obiettivo è argomentato sull'exact match. Le differenze dell'ordine del millesimo fra ablazioni vicine non sostengono alcuna conclusione, e non ne è stata tratta nessuna.

11.11 Riepilogo per non-copy accuracy

Poiché la non-copy-token accuracy è la metrica con maggiore potere discriminante su questo corpus (Capitolo 10), vale la pena chiudere con l'ordinamento che essa induce:

Sistema	Non-copy accuracy
SFT con massa ammessa	0,9685
SFT seguito da GRPO, zero-shot	0,9650
SFT soltanto	0,9510
Baseline a regole	0,9424
GRPO da modello base, few-shot	0,4692
Base zero-shot con Trie	0,0451
Base zero-shot senza Trie	0,0007

L'ordinamento è lo stesso che si ottiene con ROUGE-L, con un'eccezione significativa: le celle addestrate e la regola si stringono in alto — fra l'SFT e la regola corrono +0,0086, fra il migliore obiettivo ausiliario e la regola +0,0261 — mentre in basso le celle si separano in modo interpretabile. In particolare, il GRPO dal modello base risolve correttamente meno della metà delle posizioni non banali, contro il 94% di una tabella di corrispondenze costruita contando parole.

Nessuna cella addestrata con il solo reinforcement learning si avvicina alla regola. È il risultato del progetto, ed è negativo.

Capitolo 12

Oltre ASLG-PC12: altri dataset e altri compiti

I capitoli precedenti mostrano che su ASLG-PC12 la domanda "il GRPO aiuta?" ha poco margine per una risposta: il corpus è sintetico e una regola lo satura (sezione 3.2). Il progetto, però, non riguarda il gloss in sé: è la fase iniziale di una domanda più generale, cioè **se il GRPO possa allineare un modello a produrre uscite strutturate che rispettano vincoli rigidi, con o senza supporti** (esempi nel prompt, Trie in decodifica, SFT preliminare).

Per rispondere servono compiti in cui il margine esiste e in cui la struttura dell'uscita è diversa. Questo capitolo porta la stessa pipeline (stesso modello, stessi iperparametri GRPO, stesso disegno a celle) su tre dataset: un corpus di gloss annotato da umani, PHOENIX-2014T, e i due compiti non-gloss di GrammarRL [43], la classificazione gerarchica WOS-46985 e il riconoscimento di entità CoNLL-2003. GrammarRL valuta la decodifica vincolata da grammatica con reinforcement learning proprio su questi tre compiti; qui i dataset sono gli stessi ma i numeri non sono confrontabili uno a uno, perché split, modello, linearizzazione del target e reward sono diversi.

12.1 I tre compiti

Dataset	Compito	Ingresso	Uscita	Train / val / test
PHOENIX-2014T	traduzione in gloss	frase tedesca	gloss DGS	7.096 / 519 / 642
CoNLL-2003	entità nominate	frase inglese	`TIPO:entità`	14.041 / 3.250 / 3.453
WOS-46985	classificazione	abstract inglese	`DOMINIO Area`	$\approx$ 42.300 / — / 4.699

Tabella 12.1: I tre dataset aggiunti. PHOENIX e CoNLL usano gli split ufficiali; WOS non ne ha, e si usa la stessa logica di ASLG-PC12 (90/10 con seme 42).

Tutti e tre si scaricano da soli al primo uso, da copie su Hugging Face fissate per revisione e verificate con sha256 contro gli originali. Si usa solo il testo: di PHOENIX non si scaricano i video.

12.1.1 PHOENIX-2014T: gloss annotati da umani

PHOENIX-2014T [3] raccoglie le previsioni del tempo della televisione tedesca, con la trascrizione in tedesco e la trascrizione in gloss della lingua dei segni tedesca (DGS) fatta da annotatori umani. Un esempio reale del test set:

Tedesco	`am donnerstag regen in der nordhälfte in der südhälfte mal sonne mal wolken ähnliches wetter dann auch am freitag`
Gloss	`DONNERSTAG NORDWEST REGEN REGION SONNE WOLKE WECHSELHAFT DANN FREITAG AEHNLICH WETTER`

A differenza di ASLG-PC12 il gloss *non* è l'ingresso maiuscolizzato: le parole funzionali spariscono, l'ordine cambia, intere espressioni diventano un segno (*mal sonne mal wolken* → `SONNE WOLKE WECHSELHAFT`), le dieresi diventano digrammi (`AEHNLICH`). La sovrapposizione fra ingresso e uscita è del 15,59% contro il 98% di ASLG-PC12 (sezione 3.2). È il corpus su cui la letteratura riporta i migliori risultati intorno a ROUGE-L 55,18 [28].

Rispetto ad ASLG cambia solo il profilo del prompt (tedesco → DGS); il vocabolario chiuso del Trie conta circa 1.080 gloss estratti dal train, e tutto il resto (reward, Trie a doppia radice, metriche) è invariato.

12.1.2 CoNLL-2003: riconoscimento di entità

CoNLL-2003 [42] è il riferimento classico per il riconoscimento di entità nominate: frasi di agenzia Reuters in cui ogni entità è marcata come persona (`PER`), organizzazione (`ORG`), luogo (`LOC`) o altro (`MISC`). In GrammarRL il modello genera un JSON; qui il target è *linearizzato* in una sequenza di token `TIPO:entità`, con le parole dell'entità unite da trattini bassi, ordinate per tipo e senza ripetizioni, oppure il solo `NONE`. Esempi reali del test set:

Frase	Target
`8. Dominick Gauthier ( Canada ) 22.73`	`PER:Dominick_Gauthier LOC:Canada`
`Japan then laid siege to the Syrian ...`	`LOC:Japan MISC:Syrian`
`AL-AIN , United Arab Emirates`	`LOC:AL-AIN LOC:United_Arab_Emirates`
`on Friday :`	`NONE`

Il 20% delle frasi valutate non contiene entità. Il compito è strutturalmente diverso dal gloss: l'uscita non traduce l'ingresso, ne *seleziona* dei pezzi e li etichetta.

12.1.3 WOS-46985: classificazione gerarchica

Web of Science [20] raccoglie 46.985 abstract scientifici etichettati con un dominio (sette: informatica, ingegneria elettronica, psicologia, ingegneria meccanica, ingegneria civile, medicina, biochimica) e un'area di quel dominio, per 134 coppie (dominio, area) in tutto. Il target è linearizzato in due token: il dominio e l'area con le parole unite da trattini bassi. Un esempio reale del test set:

Abstract (inizio)	Target
`This paper is mainly discuss about software component testing as an interesting challenge in Component Based Software Engineering (CBSE) area. ...`	`CS Software_engineering`

Le etichette sono quelle canoniche del codice ufficiale delle classi. Gli abstract sono lunghi: vengono troncati alle prime 256 parole, così il prompt resta entro la lunghezza massima. Per lo stesso motivo il dataset si usa **solo in zero-shot**: tre abstract di esempio nel prompt non ci starebbero. Con due soli token, ROUGE-L e BLEU dicono poco: la metrica da leggere è l'exact match (dominio e area entrambi giusti), e il gloss-F1 dà credito parziale al solo dominio giusto.

12.2 Un vincolo per ogni compito

Il Trie a doppia radice (sezione 8.2.1) ammette qualunque sequenza di parole di un vocabolario chiuso. Va bene per il gloss, non per gli altri due compiti, e per due ragioni opposte: su CoNLL è troppo stretto, su WOS troppo largo. La grammatica dell'uscita diventa quindi una scelta del config (`grammar.mode`), e la *stessa* definizione guida il decoder, la validity e la reward di formato, così ciò che il decoder impone e ciò che le metriche chiamano "valido" non possono divergere.

- `vocab` (ASLG-PC12, PHOENIX-2014T): il Trie storico sul vocabolario di train, invariato.
- `source_spans` (CoNLL-2003): un Trie *per frase*, costruito dal prompt stesso. Ogni parola dell'uscita è `TIPO:pezzo`, dove il pezzo è una sequenza contigua di al più dieci parole della frase; in alternativa, il solo `NONE`. Con il vocabolario di train il 48% delle entità del test sarebbe stato impossibile da generare (sono nomi propri mai visti); con il Trie per frase ogni entità gold è raggiungibile e non c'è alcuna fuga di informazione dal test.
- `sequences` (WOS-46985): l'uscita intera deve essere una delle 134 coppie viste nel train. Il Trie a token libero lasciava generare `Medical Medical`, l'area prima del dominio, o un'area di un altro dominio; ora il decoder impone dominio, poi un'area di quel dominio, poi la fine.

12.2.1 Ogni entità una sola volta

La prima campagna su CoNLL ha mostrato che la grammatica `source_spans` lasciava aperto un varco, e che il GRPO lo ha trovato. Con il Trie attivo il GRPO dal modello base ha imparato a scrivere la stessa entità con tutti i tipi:

Gold	`PER:Oleg_Shatskiku`
Generato	`PER:Oleg_Shatskiku ORG:Oleg_Shatskiku LOC:Oleg_Shatskiku`

È *reward hacking* nel senso stretto: se il modello non sa il tipo, coprirsi con tutti e tre garantisce credito parziale da ROUGE-L e BLEU (qui ROUGE-L 0,5) invece di rischiare zero. Il 53% delle uscite del GRPO zero-shot ripeteva un'entità, contro il 25% del modello base e meno dell'1% dell'SFT. Le metriche lo mostrano in modo netto: ROUGE-L sale da 0,27 a 0,35, l'exact match *si dimezza* da 0,045 a 0,023, e la validity (0,998) non se ne accorge, perché l'uscita era ammessa dalla grammatica.

Una prima correzione ha vietato il *pezzo identico*: ogni pezzo della frase una sola volta, con un solo tipo. Ha funzionato solo a metà. Nella campagna rilanciata il GRPO ha trovato subito il varco accanto, scrivendo pezzi *annidati*:

Frase	`Japan then laid siege to the Syrian penalty area ...`
Gold	`LOC:Japan MISC:Syrian`
Generato	`PER:siege PER:siege_to PER:siege_to_the MISC:game`

Con la regola dei pezzi identici, i pezzi annidati erano nel 21–24% delle uscite delle celle GRPO zero-shot contro l'1,6% dell'SFT, e le uscite avevano in media 1,9 volte i token del gold; la validity restava a 0,98, perché la grammatica ammetteva tutto questo.

La correzione definitiva sta ancora nella grammatica, non nella reward: **le entità non si sovrappongono**. Ogni parola della frase può stare in più entità solo quante volte compare

nella frase (il suo *budget*). Il gold rispetta sempre il budget, perché le sue entità sono porzioni disgiunte della frase: su tutte le 20.744 frasi di train, validation e test di CoNLL ogni gold è ammesso (anche `PER:Costas_Simitis PER:Costas_Skandalidis`, dove *Costas* compare due volte nella frase), mentre `PER:siege PER:siege_to` è impossibile, perché *siege* compare una volta sola. Nel decoder ogni nodo del Trie per frase conosce i pezzi che terminano nel suo sottoalbero; un ramo nessuno dei cui pezzi sta ancora nel budget viene potato, e una parola può chiudersi solo se il suo pezzo ci sta. Così il decoder non arriva mai in un vicolo cieco. Lo stesso vincolo entra nella validity e nella reward di formato, quindi penalizza l'hack anche nelle celle senza Trie.

I risultati di CoNLL riportati più sotto sono quelli della campagna con il vincolo definitivo. Un'avvertenza sul percorso: nelle due campagne rilanciate la cella SFT→GRPO sembrava crollata (exact match 0,09). Non era il modello ma la pipeline: riusando l'adapter SFT di una run precedente della stessa cella, il train non lo copiava nella run nuova, e la valutazione caricava il modello base con il solo adapter GRPO, senza la fase SFT. Corretta la copia, le stesse run rivalutate danno 0,78; da allora la valutazione si ferma se la fase SFT attesa manca, invece di valutare in silenzio il modello sbagliato.

12.3 Adattamenti della reward

Per PHOENIX-2014T la reward è quella di ASLG-PC12 (Capitolo 7). Per CoNLL e WOS i target sono cortissimi (uno o due token: `NONE`, `PER:Mary`, `CS Machine_learning`), e i pesi di ASLG non funzionano: la componente di struttura gold vale -1 fisso sotto i due token, e la componente scalata dal verificatore, che la moltiplica, pure. Una risposta corretta di un token prendeva +0,30 invece di +1. Su questi due compiti la plausibilità dei bigrammi non porta informazione, e i due pesi passano all'ordine dei gloss (distanza di edit a token, sensibile al maiuscolo, l'unica componente che premia solo l'etichetta esatta):

Componente	ASLG / PHOENIX	CoNLL / WOS
traduzione (ROUGE-L)	0,20	0,20
BLEU	0,20	0,20
struttura gold	0,20	0
ordine dei gloss	0,10	0,40
verificatore	0,10	0
formato	0,10	0,10
ripetizione	0,10	0,10

12.4 Protocollo

Le celle sono quelle di ASLG-PC12: modello base (zero-shot, few-shot, zero-shot senza Trie), solo SFT, solo GRPO, SFT seguito da GRPO, GRPO senza Trie (few-shot e zero-shot) e SFT con obiettivo di massa ammessa. WOS ha solo le celle zero-shot. Ogni GRPO fa 5.000 passi, cioè circa 0,7 epoche di PHOENIX, 0,36 di CoNLL e 0,12 di WOS; gli SFT si fermano per early stopping. La valutazione genera cinque uscite campionate per prompt (temperatura 0,7) su tutto il test di PHOENIX (642 frasi) e su 2.000 prompt del test di CoNLL e WOS. Un solo seme per cella, come per ASLG-PC12 (sezione 11.10), salvo le sei repliche della sezione 12.5.3.

Cella	ROUGE-L	Exact match	Non-copy	Validity
Base zero-shot	0,0961	0,0000	0,1083	0,9586
Base few-shot	0,3103	0,0106	0,2923	0,9956
Base zero-shot senza Trie	0,1528	0,0000	0,0000	0,0000
SFT	0,4854	0,0184	0,3812	0,9969
GRPO zero-shot	0,2030	0,0003	0,1658	0,9997
GRPO few-shot	0,3601	0,0162	0,3325	0,9991
SFT→GRPO zero-shot	**0,5274**	**0,0283**	**0,4599**	1,0000
SFT→GRPO few-shot	0,5039	0,0240	0,4429	1,0000
GRPO senza Trie, few-shot	0,3550	0,0165	0,3339	0,9916
GRPO senza Trie, zero-shot	0,2898	0,0000	0,0000	0,0000
SFT con massa ammessa	0,4819	0,0178	0,3709	0,9981

Tabella 12.2: PHOENIX-2014T, test completo.

12.5 Risultati

12.5.1 PHOENIX-2014T

Su PHOENIX il margine c'è, e nessun sistema si avvicina al tetto: l'exact match resta sotto il 3% ovunque. Qui, a differenza di ASLG, il GRPO dopo l'SFT aggiunge qualcosa di visibile: ROUGE-L da 0,4854 a 0,5274 e non-copy da 0,3812 a 0,4599, dello stesso ordine dei migliori valori in letteratura [28] (con implementazioni di ROUGE diverse, per cui il confronto è solo indicativo). Il GRPO dal modello base impara (ROUGE-L da 0,0961 a 0,2030 in zero-shot, da 0,3103 a 0,3601 in few-shot) ma resta sotto l'SFT. La massa ammessa, che su ASLG era la cella migliore, qui vale quanto l'SFT semplice.

La cella GRPO senza Trie zero-shot ha ROUGE-L 0,29 con validity zero: il modello scrive parole tedesche minuscole separate da virgole (`regen, schnee, nacht, nach, nordosten, schauer`), che ROUGE-L, insensibile al maiuscolo, premia per accidente (sezione 10.1).

Il Trie costruito su tutto il dataset. Come su ASLG-PC12, una cella GRPO few-shot ha il vocabolario del Trie estratto anche dal test (una fuga di informazione deliberata, per misurare quanto costa il tetto del vocabolario di train). Su PHOENIX non costa nulla: ROUGE-L 0,3575 contro 0,3601 della cella normale, exact match identico (0,0162), validity 0,998. Il vocabolario di train non è un limite su questo corpus.

Un modello tre volte più grande. Le celle principali sono state ripetute con Qwen2.5-1.5B-Instruct, cambiando solo il modello:

	ROUGE-L		Exact match	
Cella	0,5B	1,5B	0,5B	1,5B
Base zero-shot	0,0961	0,1199	0,0000	0,0000
Base few-shot	0,3103	0,3519	0,0106	0,0131
SFT	0,4854	0,5015	0,0184	0,0171
SFT→GRPO zero-shot	0,5274	0,5421	0,0283	0,0318
GRPO few-shot	0,3601	0,4337	0,0162	0,0221

Tabella 12.3: PHOENIX-2014T, Qwen2.5 da 0,5B e da 1,5B a parità di tutto il resto.

Il modello più grande aiuta poco: +1–2 punti di ROUGE-L sulle celle con SFT, e l'exact match resta sotto il 3,2% in ogni cella. Il guadagno maggiore è nel GRPO few-shot dal modello base (+7 punti di ROUGE-L, non-copy da 0,33 a 0,39). Su PHOENIX il limite non

è la dimensione del modello, e il SFT→GRPO resta la cella migliore a entrambe le taglie. La baseline zero-shot da 1,5B ha validity più bassa (0,80 contro 0,96): sotto Trie il modello più grande si ripete più spesso fino al limite di lunghezza.

12.5.2 CoNLL-2003

Cella	Exact match	Gloss-F1	ROUGE-L	Validity
Base zero-shot	0,0445	0,0816	0,2722	0,9987
Base few-shot	0,0714	0,1845	0,3823	0,9992
Base zero-shot senza Trie	0,0011	0,0022	0,2676	0,0075
SFT	0,7448	0,8255	0,9005	1,0000
GRPO zero-shot	0,0296	0,1298	0,3167	0,9998
GRPO few-shot	0,3247	0,3776	0,5396	0,9997
SFT→GRPO zero-shot	**0,7811**	**0,8546**	**0,9148**	1,0000
SFT→GRPO few-shot	0,7159	0,8149	0,8852	1,0000
GRPO senza Trie, few-shot	0,3476	0,3776	0,6113	0,6501
GRPO senza Trie, zero-shot	0,1472	0,0732	0,4027	0,4578
SFT con massa ammessa	0,7505	0,8253	0,9044	1,0000

Tabella 12.4: CoNLL-2003, 2.000 prompt del test, con il vincolo delle entità non sovrapposte (sezione 12.2.1). Gloss-F1 è il micro-F1 sui token `TIPO:entità`, cioè l'F1 a livello di entità.

La gerarchia è netta: l'SFT porta l'exact match da 0,04 a 0,74, il GRPO dopo l'SFT aggiunge quasi quattro punti in zero-shot (0,7811, lo stesso valore della prima campagna senza vincolo: alla policy SFT il vincolo non toglie nulla), il GRPO dal modello base resta lontano.

Con il vincolo l'hack non c'è più, ma il GRPO zero-shot dal modello base non impara comunque il compito (exact match 0,030). Separando le due metà del compito si vede dove fallisce: gli *span* li trova (richiamo 0,66 contro 0,50 del modello base, ignorando il tipo), i *tipi* no. Emette `MISC` 15.004 volte contro le 1.990 del gold, e sulle entità tipate la precisione è 0,09. Il segnale per imparare il tipo è debole: ROUGE-L spezza `LOC:Japan` in due parole, e con tutti gli span giusti ma i tipi sbagliati dà ancora fra +0,14 e +0,43 (contro −1 di `NONE`); tutti `MISC` rende più di tutti `ORG`. In few-shot, dove gli esempi mostrano i tipi, l'exact match sale a 0,32.

Una reward che guarda il tipo, risultato negativo. Se il segnale sui tipi è debole, la correzione naturale è una reward che li distingua. Due celle hanno sostituito ROUGE-L e BLEU (peso totale 0,40) con l'F1 fra i token di uscita e di gold, uno per entità tipata: lo span giusto col tipo sbagliato vale zero.

	Exact match		Gloss-F1	
GRPO dal modello base	ROUGE+BLEU	entity-F1	ROUGE+BLEU	entity-F1
zero-shot	0,030	0,042	0,130	0,102
few-shot	0,325	0,334	0,378	0,377

La reward nuova non cambia il risultato. In zero-shot la componente di entity-F1 parte da −0,88 e a fine addestramento è a −0,84, e il 42% dei gruppi ha varianza nulla: il modello base non azzecca quasi mai un'entità tipata, quindi quasi ogni candidato vale −1 e non c'è nulla da confrontare dentro il gruppo. Il modello emette anzi `MISC` ancora più spesso (24.152 volte contro 15.004). In few-shot non cambia nulla. Il collo di bottiglia non è la reward ma l'*esplorazione*: dal modello base il GRPO non campiona abbastanza tipi giusti da poterli

rinforzare. È lo stesso risultato della sezione 12.6, visto da un'altra parte. [DA FARE: aggiungere GRPO zero-shot con Qwen2.5-1.5B su CoNLL (in corso): dice se un modello più capace scopre i tipi]

12.5.3 Quanto pesano i semi

Ogni cella del progetto ha un solo seme, e a seme fisso l'addestramento è deterministico (sezione 11.10): la variabilità fra semi non è mai stata misurata. Sei celle sono state replicate con un secondo seme di addestramento (cambia solo l'RNG del GRPO; split, prompt valutati e, per SFT→GRPO, la fase SFT restano quelli della cella originale):

Dataset	Cella	seme 42	seme 1	SFT
CoNLL-2003	SFT→GRPO zero-shot, exact match	0,781	0,787	0,745
WOS-46985	SFT→GRPO zero-shot, exact match	0,787	0,785	0,762
PHOENIX-2014T	SFT→GRPO zero-shot, exact match	0,028	0,023	0,018
PHOENIX-2014T	SFT→GRPO zero-shot, ROUGE-L	0,527	0,522	0,485
CoNLL-2003	GRPO zero-shot, exact match	0,030	0,038	—
CoNLL-2003	GRPO senza Trie, zero-shot, validity	0,458	0,202	—
PHOENIX-2014T	GRPO senza Trie, few-shot, validity	0,992	0,994	—

Tre risultati. Primo, **il guadagno di SFT→GRPO sull'SFT è stabile** su CoNLL e WOS: le due repliche differiscono di meno di un punto e restano entrambe sopra l'SFT di 2,3–4,3 punti. Su PHOENIX l'exact match è troppo basso perché i soli punti decimali dicano qualcosa, ma ROUGE-L (+3,7 e +4,0 punti) e gloss-F1 reggono. Secondo, **con il Trie le celle sono stabili**: anche il GRPO zero-shot dal modello base differisce di meno di un punto. Terzo, **senza Trie dipende**: con gli esempi nel prompt (PHOENIX few-shot) la validity è 0,99 a entrambi i semi, mentre su CoNLL zero-shot passa da 0,46 a 0,20 e l'exact match da 0,15 a 0,06. Quando la struttura non è garantita dal decoder né dal prompt, che il GRPO la trovi o meno dipende dal caso dei primi rollout; quel singolo valore va letto come un ordine di grandezza, non come una misura.

12.5.4 WOS-46985

Cella	Exact match	Gloss-F1	Validity
Base zero-shot	0,1039	0,2876	1,0000
Base zero-shot senza Trie	0,0000	0,0146	0,0000
SFT	0,7622	0,8269	1,0000
GRPO zero-shot	0,1688	0,3702	1,0000
SFT→GRPO zero-shot	**0,7869**	**0,8475**	1,0000
GRPO senza Trie	0,0000	0,1467	0,0001
SFT con massa ammessa	0,7572	0,8255	1,0000

Tabella 12.5: WOS-46985, 2.000 prompt del test, solo zero-shot.

Con il Trie a sequenze il modello base è sempre valido ma sceglie quasi a caso fra poche classi: `ECE System_identification` e `Psychology Social_cognition` coprono da sole un terzo delle sue uscite. Il GRPO migliora di sei punti (0,1039 → 0,1688) senza uscire da questo regime; l'SFT porta l'exact match a 0,76, e il GRPO dopo l'SFT aggiunge due punti e mezzo. Senza Trie il modello non produce mai una coppia valida: scrive `Medical Psychology, Medical Psychology`, con dominio e area presi dalla stessa lista e una virgola.

12.6 Che cosa dicono i tre dataset insieme

12.6.1 Senza Trie e senza esempi, il GRPO trova il formato a fatica

È il risultato più diretto per la domanda del progetto. Senza Trie, la validity delle celle GRPO dipende soprattutto dalla presenza di esempi nel prompt:

GRPO senza Trie	few-shot	zero-shot
ASLG-PC12	0,99	—
PHOENIX-2014T	0,99 (0,99 col seme 1)	0,00
CoNLL-2003	0,65	0,20–0,46
WOS-46985	—	0,0001

Il meccanismo si legge nei log di addestramento. Su PHOENIX zero-shot la componente di formato della reward resta fra -1 e $-0{,}99$ per tutti i 5.000 passi: nessun candidato ha mai il formato giusto, quindi quasi nessun gruppo contiene un confronto fra uscita valida e non valida, e il formato non riceve gradiente. Su WOS qualche candidato valido compare per caso e la componente sale (da $-0{,}98$ a $-0{,}12$), ma non abbastanza da rendere valide le uscite in valutazione. Su CoNLL, dove il formato `TIPO:entità` è più vicino a ciò che il modello scrive spontaneamente, la componente sale di più (da $-0{,}93$ a $-0{,}28$ e da $-0{,}87$ a $-0{,}35$ nelle due repliche) e la validity arriva a metà, ma con una variabilità fra semi che la rende instabile (sezione 12.5.3). Il GRPO, dunque, **rinforza una struttura che il modello già produce qualche volta; non la scopre**. Gli esempi nel prompt, il Trie o l'SFT sono i supporti che la rendono presente fin dall'inizio, e senza almeno uno dei tre la struttura non arriva.

12.6.2 Con un vincolo attivo, il GRPO cerca il varco

L'hack di CoNLL (sezione 12.2.1) è il rovescio della stessa medaglia: quando la grammatica lascia un'uscita ammessa che la reward premia più del dovuto, il GRPO la trova, e la validity non lo segnala perché l'uscita è formalmente corretta. Su CoNLL è successo due volte di fila: chiuso il varco dei tipi ripetuti, il GRPO ha aperto quello delle entità annidate. Un vincolo rigido va quindi scritto pensando a che cosa *non* deve permettere, non solo a che cosa deve garantire. E chiudere il varco non basta a far imparare il compito: tolto l'hack, il GRPO zero-shot resta a un exact match di 0,03.

12.6.3 SFT→GRPO è la cella migliore ovunque, ma il guadagno è piccolo

Su tutti e tre i dataset la cella migliore è l'SFT seguito dal GRPO in zero-shot, con un margine di due-quattro punti di exact match sull'SFT. Come su ASLG, il segnale di apprendimento si esaurisce: alla fine dell'addestramento la frazione di gruppi a varianza nulla è l'83% su CoNLL e il 77% su WOS (su PHOENIX, dove la policy è più lontana dal tetto, solo il 7%). La massa ammessa, migliore cella su ASLG-PC12, sugli altri tre dataset vale quanto l'SFT semplice.

Cella	Reward media inizio	fine	Gruppi a varianza nulla inizio	fine
PHOENIX GRPO zero-shot	−0,60	−0,39	0%	0%
PHOENIX SFT→GRPO zero-shot	−0,06	0,10	0%	7%
CoNLL GRPO zero-shot	−0,47	−0,37	24%	26%
CoNLL SFT→GRPO zero-shot	0,76	0,82	74%	83%
WOS GRPO zero-shot	−0,38	−0,21	34%	36%
WOS SFT→GRPO zero-shot	0,56	0,68	58%	77%

Tabella 12.6: Medie sui primi e sugli ultimi 50 passi registrati di ogni addestramento GRPO.

Capitolo 13

Organizzazione del codice

Questo capitolo descrive la struttura del codice e i meccanismi che garantiscono la coerenza fra le celle sperimentali. Serve come mappa per chi voglia riprodurre o estendere il lavoro.

13.1 Struttura dei moduli

Percorso	Responsabilità
`src/datasets/`	caricamento, suddivisione, retrieval, transizioni
`src/models/`	caricamento del modello, teste aggiuntive
`src/grammar/`	Trie, maschera, processore di logits
`src/rewards/t2g_rewards.py`	le componenti di reward
`src/training/`	punti di ingresso e trainer (SFT, GRPO, ausiliari)
`src/analysis/`	baseline a regole, diagnostiche
`src/utils/`	metriche, configurazione, prompt, monitoraggio
`cluster/`	script SLURM e libreria di shell
`remote/`	servizio FastAPI e interfaccia testuale
`tests/`	suite di test
`experiments/configs/qwen25-05b/`	configurazioni delle celle

Il pacchetto `src/grammar/` è indipendente dal modello: Trie e maschera sono strutture dati pure, verificabili senza GPU. Il pacchetto `src/analysis/` contiene codice che non partecipa all'addestramento ma è essenziale all'interpretazione; la baseline a regole vive qui, non fra i modelli, perché non è un modello ma uno strumento di misura.

13.2 Il sistema di configurazione

Ogni cella sperimentale è descritta da un file YAML. Il meccanismo di composizione è la chiave `extends`, risolta da `src/utils/config.py` (funzione `resolve_config`), che esegue una *fusione profonda ricorsiva*: i dizionari annidati vengono fusi chiave per chiave invece di essere sostituiti in blocco, le liste e gli scalari del figlio sostituiscono quelli del padre, e i percorsi dei genitori sono relativi al file figlio. Una cella dichiara così soltanto le proprie differenze rispetto a `base.yaml`, e ogni modifica al file di base si propaga a tutte. La chiave `extends` è rimossa dalla configurazione risolta: i trainer ricevono una configurazione completa e piatta.

13.3 Costruzione centralizzata del prompt

La funzione `build_t2g_prompt` in `src/utils/prompting.py` è l'unica sorgente del template del prompt, usata in modo identico da addestramento, valutazione e retrieval. Il gloss di riferimento, durante addestramento e valutazione, è ricercato tramite l'hash SHA256 dell'istruzione dell'utente e non tramite la stringa letterale del prompt: in questo modo una differenza di formattazione non produce corrispondenze mancate silenziose, che azzererebbero il ROUGE-L senza alcun errore.

13.4 La matrice sperimentale

Le celle sono raggruppate per tipo. Le tre celle `baseline` sono di sola valutazione e misurano il modello non addestrato.

Gruppo	Cella	Tipo
`baseline`	`zero-shot`	sola valutazione
	`zero-shot-no-grammar`	sola valutazione
	`few-shot`	sola valutazione
`sft`	`zero-shot`	addestramento
	`zero-shot-greedy`	sola valutazione
`grpo`	`zero-shot`, `few-shot`	addestramento
`sft-grpo`	`zero-shot`, `few-shot`	addestramento
`ablations/rewards`	`edit-validity`	addestramento
`ablations/loss`	`dr-grpo`	addestramento
`ablations/decoding`	`no-grammar`, `hot-rollout`	addestramento
`ablations/objectives`	`sft-allowed-mass`	addestramento
	`sft-allowed-mass-greedy`	sola valutazione
	`sft-structured`	addestramento
	`sft-structured-shuffled`	addestramento

Le celle di ablazione isolano ciascuna un fattore che la parte principale dell'esperimento confonde:

- `edit-validity` varia la composizione dello stack di reward (Capitolo 7);
- `dr-grpo` attiva la variante di denominatore della loss che le altre celle non selezionano;
- `no-grammar` isola il contributo del vincolo simbolico (Capitolo 8);
- `hot-rollout` varia la temperatura di generazione dei candidati, cioè il parametro che governa il supporto dei rollout;
- le celle in `objectives` corrispondono ai due obiettivi ausiliari del Capitolo 9, la seconda con il controllo a grafo permutato.

Le coppie `zero-shot`/`few-shot` differiscono per la modalità di prompting, che comporta due parametri solidali: `retrieval.enabled` (con $k = 3$) e `max_prompt_length`, portato da 256 a 768, perché gli esempi recuperati allungano il prompt e un limite troppo basso li troncherebbe in silenzio.

13.5 La validazione delle configurazioni

Lo script `tests/validate_configs.py` risolve ogni configurazione sperimentale e ne verifica le invarianti: presenza delle sezioni attese, tipi dei valori, somma dei pesi di reward pari a 1,0, rifiuto delle chiavi non previste, e `max_prompt_length` ≥ 512 in presenza di retrieval.

13.6 Le diagnostiche di Markov

Il modulo `src/analysis/markov_diagnostics.py` fornisce strumenti di ispezione sui percorsi di transizione fra gloss (punteggio logaritmico di un percorso, percorso di massima verosimiglianza, versione smussata). Sono validati contro un calcolo a forza bruta su istanze piccole e rifiutano spazi di stato superiori a 256 elementi: servono a ispezionare casi specifici, non a ottimizzare. Le stesse quantità, usate come reward, sono state rimosse perché ridondanti (sezione 7.2) e conservate come sola diagnostica.

13.7 Ambiente di esecuzione

Il cluster è gestito da SLURM con una quota di una sola allocazione attiva per utente: le celle sperimentali sono quindi eseguite rigorosamente in sequenza, tramite una coda persistente fatta avanzare da attivazioni periodiche. I nodi di calcolo sono privi di accesso alla rete, per cui modello e dati sono scaricati in anticipo in una cache locale da un passo di preparazione, e ogni job parte con l'ambiente offline di HuggingFace impostato prima del primo import Python.

13.8 Il servizio remoto

La directory `remote/` contiene un servizio FastAPI e un'interfaccia testuale che pilotano il cluster a distanza: sottomettere celle, ispezionare la coda, recuperare i risultati.

13.9 La suite di test

La maggior parte dei test verifica componenti eseguibili senza GPU: le strutture simboliche (Trie, maschera, insieme ammesso), le componenti di reward su candidati costruiti a mano, comprese quelle avversarie, le metriche, i contratti degli obiettivi ausiliari (inerzia esatta a peso nullo e rifiuto delle configurazioni incompatibili) e le guardie dei punti di ingresso. I test che richiedono il download del dataset sono separabili dal resto.

Capitolo 14

Conclusioni

14.1 Che cosa è stato stabilito

Il lavoro non produce un miglioramento delle metriche. Produce affermazioni sostenute da misure, che conviene enunciare nella forma più compatta.

14.1.1 Le metriche su ASLG-PC12 sono sature

Una baseline a regole di poche righe, che non apprende nulla, ottiene ROUGE-L 0,9685 ed exact match 0,5865. È il riferimento obbligato per leggere qualsiasi numero del lavoro. La causa è la natura sintetica del corpus (sezione 3.2): il gloss è quasi una trasformazione deterministica del testo. La metrica stessa contribuisce, perché l'implementazione di ROUGE-L impiegata è insensibile al maiuscolo e ai caratteri non alfanumerici (sezione 10.1).

Ne segue un risultato che rafforza la tesi: l'SFT (ROUGE-L 0,9681) *non* supera la baseline a regole su ROUGE-L. Il suo unico vantaggio reale è l'exact match (0,7662 contro 0,5865). Dove ROUGE-L è saturo, la metrica che discrimina è quella più severa.

14.1.2 Che cosa compra il vincolo simbolico

Il Trie (Capitolo 8) fa *crollare* ROUGE-L del modello base da 0,3648 a 0,1373, e contemporaneamente porta la validity da 0,0370 a 0,9055 e la non-copy-token accuracy da 0,0007 a 0,0451. Il vincolo sposta quindi il modello da uscite fuori vocabolario, che la sovrapposizione lessicale premia per accidente, a uscite valide che il modello non sa ancora comporre. Con il prompt few-shot, il modello vincolato raggiunge ROUGE-L 0,4664 e validity 0,9821: quasi tutto il guadagno di questo passaggio (+0,33) viene dal prompting, non dall'aggiornamento di parametri.

Soprattutto, il vincolo fa ciò che il reinforcement learning da solo non fa: garantisce la struttura. Senza Trie il GRPO impara l'aspetto di un gloss e la validity permissiva sale a 0,99, ma solo il 19% delle uscite è fatto interamente di gloss del vocabolario; con il Trie la quota è fra il 98% e il 100%, e il residuo sono uscite troncate al limite di lunghezza con l'ultimo gloss tagliato. Il vincolo *non* crea invece il segnale di apprendimento: i gruppi a varianza nulla sono meno del 6% con o senza Trie. Il costo che il vincolo impone sulla qualità del contenuto è documentato in letteratura [37, 55, 50]; una sua valutazione va fatta sulla validità stretta, non sul solo ROUGE-L.

14.1.3 La posizione onesta sul GRPO

La domanda "il GRPO migliora la generazione di gloss?" ha su ASLG-PC12 un margine molto ridotto in cui rispondere, perché la baseline a regole e l'SFT sono già in saturazione. I dati sostengono soltanto quanto segue.

Partendo dal modello base, il GRPO migliora molto (ROUGE-L 0,6082 con few-shot e 0,5173 con zero-shot, contro 0,4664 e 0,1373 dei rispettivi punti di partenza vincolati), ma resta ben al di sotto dell'SFT e della baseline a regole. Applicato a una policy SFT già convergente, il GRPO non produce né un miglioramento sostanziale né un degrado: SFT→GRPO zero-shot (0,9744 / 0,8095) è sopra l'SFT semplice (0,9681 / 0,7662), la variante few-shot (0,9628 / 0,7296) è di poco sotto, e le differenze sono entro frazioni di punto in entrambe le direzioni. Poiché l'addestramento a seme fisso è deterministico bit per bit, questo lavoro non dispone di alcuna stima della variabilità fra run: le differenze di questa entità non consentono di concludere alcunché sulla direzione dell'effetto.

14.1.4 Gli obiettivi ausiliari

I due obiettivi del Capitolo 9 hanno esiti opposti (sezione 11.8). La **massa ammessa** migliora l'SFT: ROUGE-L 0,9769 ed exact match 0,8341, contro 0,9681 e 0,7662; in valutazione greedy 0,9774 e 0,8420, per cui l'effetto non dipende dal campionamento. È l'unico candidato da portare avanti. La **loss strutturata** fallisce il proprio controllo negativo: la cella con grafo permutato (0,9464 / 0,6078) è pari o migliore di quella con il grafo vero (0,9430 / 0,6053), ed entrambe sono peggiori dell'SFT. L'informazione sulle transizioni non contribuisce, e l'obiettivo va abbandonato.

14.1.5 Che cosa aggiungono gli altri dataset

Su PHOENIX-2014T, CoNLL-2003 e WOS-46985 (Capitolo 12) la gerarchia delle celle è la stessa di ASLG-PC12: l'SFT fa quasi tutto, il GRPO dopo l'SFT aggiunge due-quattro punti di exact match, il GRPO dal modello base resta lontano. Due risultati sono nuovi e riguardano direttamente la domanda sui vincoli. Primo: **senza Trie il GRPO impara il formato solo se il prompt contiene esempi** (validity 0,99 few-shot contro 0 zero-shot su PHOENIX; su CoNLL zero-shot al più a metà, e in modo instabile fra semi); se nessun candidato è valido, la reward di formato è costante e non dà gradiente. Secondo: **con un vincolo attivo il GRPO sfrutta i varchi della grammatica**: su CoNLL ha imparato a ripetere ogni entità con tutti i tipi, dimezzando l'exact match mentre ROUGE-L saliva, e bloccato quello a scrivere entità annidate. La correzione è stata un vincolo più stretto, non una reward diversa; tolto l'hack, però, il GRPO dal modello base non impara i tipi. Né li impara con una reward che premia solo il tipo giusto (F1 sulle entità tipate): il collo di bottiglia è l'esplorazione, non il segnale, perché il modello base non campiona abbastanza tipi giusti da poterli rinforzare. Le repliche con un secondo seme confermano il guadagno di SFT→GRPO sull'SFT (CoNLL e WOS) e la stabilità delle celle con Trie; su PHOENIX, un modello tre volte più grande (1,5B) non porta l'exact match sopra il 3,2%.

14.2 Lavoro futuro

14.2.1 Cambiare corpus

Il passo su PHOENIX-2014T è stato fatto (Capitolo 12): il margine esiste, ma con un modello da mezzo miliardo di parametri l'exact match resta sotto il 3%, e il prossimo passo naturale è un modello più grande. Una direzione più radicale, coerente con la critica al gloss come

rappresentazione [52, 51], è abbandonare l'intermedio testuale, applicando il rinforzo alla traduzione diretta con segnali di qualità visivi [36].

14.2.2 Ridisegnare l'esperimento primario

L'esperimento primario dovrebbe essere una griglia prompting × decodifica, con il metodo di addestramento come fattore secondario: sono i due fattori con l'effetto misurato più grande, e il disegno attuale li confonde. Il metodo di addestramento va applicato dopo aver identificato la combinazione di prompting e decodifica più favorevole.

14.2.3 Consolidare la massa ammessa

Prima di considerare stabilito il risultato della massa ammessa servono più seed, poiché l'unica replica disponibile non permette una stima della variabilità, e una sonda in inferenza sulla massa rimossa che ne verifichi il meccanismo.

14.2.4 Il reinforcement learning, se e quando

Prima di addestrare con reinforcement learning conviene misurare se il segnale esiste: si generano G candidati su un campione di prompt, si calcolano le reward e si contano i gruppi a varianza nulla. Se la frazione è alta, il reinforcement learning non apprenderà indipendentemente dagli iperparametri. La misura richiede minuti.

14.3 La lezione trasferibile

> Prima di ottimizzare una metrica, misurare quanto la si può ottenere senza apprendimento; e prima di addestrare con reinforcement learning, misurare se il segnale esiste.

La prima misura, qui una tabella di regole contata sul training set, ha rivelato ROUGE-L 0,9685 e ha reso interpretabile ogni numero successivo.

Bibliografia

[1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.

[2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding LLMs the right way: Fast, non-invasive constrained generation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[3] Necati Cihan Camgoz, Simon Hadfield, Oscar Koller, Hermann Ney, and Richard Bowden. Neural sign language translation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[4] Leshem Choshen, Lior Fox, Zohar Aizenbud, and Omri Abend. On the weaknesses of reinforcement learning for neural machine translation. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020. arXiv:1907.01752.

[5] Timothee Cour, Ben Sapp, and Ben Taskar. Learning from partial labels. *Journal of Machine Learning Research*, 12:1501–1536, 2011.

[6] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. In *International Conference on Learning Representations (ICLR)*, 2021.

[7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.

[8] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. *arXiv preprint arXiv:2210.10760*, 2022.

[9] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. *arXiv preprint arXiv:2305.13971*, 2023.

[10] Kartik Goyal, Chris Dyer, and Taylor Berg-Kirkpatrick. An empirical investigation of global and local normalization for recurrent neural sequence models. *arXiv preprint arXiv:1904.06834*, 2019.

[11] Daya Guo et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[12] Kelvin Guu, Panupong Pasupat, Evan Zheran Liu, and Percy Liang. From language to programs: Bridging reinforcement learning and maximum marginal likelihood. *arXiv preprint arXiv:1704.07926*, 2017.

[13] Sil Hamilton and David Mimno. Lost in space: Finding the right tokens for structured output. *arXiv preprint arXiv:2502.14969*, 2025.

[14] Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. *arXiv preprint arXiv:1704.07138*, 2017.

[15] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.

[16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[17] Huang et al. On the effectiveness of globally normalized models for sequence labeling. *arXiv preprint arXiv:2106.03376*, 2021.

[18] Samuel Kiegeland and Julia Kreutzer. Revisiting the weaknesses of reinforcement learning for neural machine translation. *arXiv preprint arXiv:2106.08942*, 2021.

[19] Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.

[20] Kamran Kowsari, Donald E. Brown, Mojtaba Heidarysafa, Kiana Jafari Meimandi, Matthew S. Gerber, and Laura E. Barnes. HDLTex: Hierarchical deep learning for text classification. In *Proceedings of the 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2017.

[21] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

[22] Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. ProRL: Prolonged reinforcement learning expands reasoning boundaries in large language models. *arXiv preprint arXiv:2505.24864*, 2025.

[23] Zichen Liu et al. Understanding R1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[24] Amit Moryossef et al. An open-source gloss-based baseline for spoken to signed language translation. *arXiv preprint arXiv:2305.17714*, 2023.

[25] Amit Moryossef, Kayo Yin, Graham Neubig, and Yoav Goldberg. Data augmentation for sign language gloss translation. *arXiv preprint arXiv:2105.07476*, 2021.

[26] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999.

[27] Achraf Othman and Mohamed Jemni. English-ASL gloss parallel corpus 2012: ASLG-PC12. In *Proceedings of the 5th Workshop on the Representation and Processing of Sign Languages, LREC*, 2012.

[28] Younes Ouargani and Noussaima El Khattabi. Advancing text-to-GLOSS neural translation using a novel hyper-parameter optimization technique. *arXiv preprint arXiv:2309.02162*, 2023.

[29] Long Ouyang et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[30] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002.

[31] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. Grammar-aligned decoding. *arXiv preprint arXiv:2405.21047*, 2024.

[32] Yuqi Peng, Jian Zeng, and Hai Zhao. Better sign language translation with monolingual data. *arXiv preprint arXiv:2304.10844*, 2023.

[33] Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[34] Maja Popović. chrF: Character n-gram f-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, 2015.

[35] Matt Post and David Vilar. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. *arXiv preprint arXiv:1804.06609*, 2018.

[36] Rao et al. RVLF: A reinforcing vision-language framework for gloss-free sign language translation. *arXiv preprint arXiv:2512.07273*, 2025.

[37] Reddy, Walker, Ide, and Bedi. The hidden cost of structured generation in LLMs: Draft-conditioned constrained decoding. *arXiv preprint arXiv:2603.03305*, 2026.

[38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[39] Junghoon Seo and Joon Suk Huh. On the power of deep but naive partial label learning. *arXiv preprint arXiv:2010.11600*, 2020.

[40] Zhihong Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[41] Prasann Singhal, Tanya Goyal, Jiacheng Xu, and Greg Durrett. A long way to go: Investigating length correlations in RLHF. *arXiv preprint arXiv:2310.03716*, 2023.

[42] Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of the Seventh Conference on Natural Language Learning (CoNLL)*, 2003.

[43] Gabriele Tuccio, Antonino Furnari, Aldo Gangemi, and Misael Mongiovì. GrammarRL: Effective grammar-constrained decoding via reinforcement learning. *arXiv preprint arXiv:2609.39869*, 2026.

[44] Harry Walsh, Ben Saunders, and Richard Bowden. Select and reorder: A novel approach for neural sign language production. *arXiv preprint arXiv:2404.11532*, 2024.

[45] Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training. *arXiv preprint arXiv:1908.04319*, 2019.

[46] Brandon T. Willard and R'emi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[47] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[48] Fang Wu, Weihao Xuan, Ximing Lu, Mingjie Liu, Yi Dong, Zaid Harchaoui, and Yejin Choi. The invisible leash? why RLVR may or may not escape its origin. *arXiv preprint arXiv:2507.14843*, 2026.

[49] Xingzi Xu and Karim Bouyarmane. Trie automata for constrained decoding over large finite sets. *arXiv preprint arXiv:2608.12574*, 2026.

[50] Ye et al. Efficient and asymptotically unbiased constrained decoding for large language models. *arXiv preprint arXiv:2504.09135*, 2025.

[51] Kayo Yin, Amit Moryossef, Julie Hochgesang, Yoav Goldberg, and Malihe Alikhani. Including signed languages in natural language processing. *arXiv preprint arXiv:2105.05222*, 2021.

[52] Kayo Yin and Jesse Read. Better sign language translation with STMC-transformer. *arXiv preprint arXiv:2004.00588*, 2020.

[53] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

[54] Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. Filtered semi-markov CRF. *arXiv preprint arXiv:2311.18028*, 2023.

[55] Zhou. From hallucination to structure snowballing: The alignment tax of constrained decoding in LLM reflection. *arXiv preprint arXiv:2604.06066*, 2026.